Bachelor Thesis

Nsak as Framework for Scenario Based Network Security

Course of study	Bachelor of Science in Computer Science
Author	Frank Gauss (gausf1) and Lukas von Allmen (vonal3)
Advisor	Wenger Hansjürg

Version 1.0 of April 2, 2026

Technik und Informatik

Abstract

One-paragraph summary of the entire study – typically no more than 250 words in length (and in many cases it is well shorter than that), the Abstract provides an overview of the study.

Contents

1. Introduction

Computer networks are constantly exposed to evolving cyber threats ...

The *Network Swiss Army Knife (NSAK)* is a framework designed to execute orchestrated security scenarios consisting of reusable attack drills in a controlled network environment. The framework aims to support automated security testing and adversary emulation on network edge devices.

The development of the NSAK framework originates from previous project work, in which the core architecture and the basic scenario execution model were developed. This thesis builds upon this foundation and extends the framework by investigating how AI agents can support automated red and blue team scenarios.

...

1.1. Motivation

Hypothesis: AI can enhance red and blue team simulations in a network environment by generating and executing scenarios within the NSAK framework and assisting in the evaluation of detection results.

1.2. Predecessor Project: “Project 2”

Describe what we did in project 2 and what project 2 is (BFH module, possible predecessor to bachelor thesis)

1.3. Project Goals

For a goal to be considered **S.M.A.R.T.**, it must fulfill the following criteria:

- **Specific**: What do we want to achieve?
- **Measurable**: What are the success criteria?
- **Achievable**: How do we plan to achieve this goal?
- **Relevant**: Why is it important to achieve this goal?

- **Time-bound**: Goals categorized as *must* have to be achieved within the thesis, while *should* or *could* goals are considered optional or nice to have.

1.3.1. Research Goals

Compare Frameworks (must)

To identify the USP (unique selling point) of the NSAK framework, we will analyze and compare existing security emulation frameworks. First, relevant frameworks such as MITRE Caldera and Atomic Red Team will be identified and reviewed. Based on this analysis, the identified concepts will be compared with the architecture of the NSAK framework. The comparison will focus on how scenarios and drills are defined, how configuration and parameter handling are structured, how reusable components are organized, how automation is achieved, and which AI capabilities they provide.

The analysis and comparison will be based on the following properties:

- Concepts
- Modularity
- Configurability
- Automation
- AI Capabilities

Success criteria:

- At least two security emulation frameworks are identified and reviewed
- A matrix comparing the specified properties of the selected frameworks is created
- The USP of the NSAK framework is identified

AI Integration (must)

The goal of this research is to investigate how AI (artificial intelligence) can support automated security testing and adversary emulation. The research evaluates potential integration points where AI could enhance the scenario execution, automation, decision-making, and analysis capabilities of the NSAK framework.

Success criteria:

- How is AI currently used in cybersecurity automation?
- Which tasks in adversary emulation can be supported or automated by AI?
- How could AI interact with the NSAK scenarios and drills (e.g., Agents, MCP)?
- What architectural changes would be required to integrate AI into NSAK?

1.3.2. Framework

Configuration Management (must)

The goal is to extend the NSAK framework, so that scenarios and drills can expose configurable parameters to increase the flexibility of scenarios and reusability of drills. Parameters can be provided via command-line arguments or YAML files and are resolved by the framework into a unified run configuration used for scenario execution.

Success criteria:

- Scenarios and drills can expose configurable parameters
- The available parameters can be listed by the CLI with the `--help` option
- Parameters can be provided via CLI arguments or YAML files
- The framework resolves the inputs into a run configuration (internal data structure).

REST API (TBD)

Adding a REST API to the NSAK framework was already proposed in the predecessor project, as it greatly enhances interoperability with other tools. The priority of this goal depends heavily on the outcome of the goal 1.3.1 and on whether we want to fulfill the goal 1.3.2, which is prioritized as “could”. If the evaluated AI integration requires a REST API (e.g., MCP), this goal needs to be prioritized as “must”; it will be prioritized as “could”.

Success criteria:

- A REST API specification with feature parity to the CLI is created.

- The REST API is implemented according to the specification.
- The API is documented (Open API) and usable by external clients.
- The REST API remains optional and does not replace the CLI-based interaction.
- Authentication and Authorization are explicitly out of scope.

MCP Server (TBD)

The MCP Protocol allows the AI to interact with applications in a standardized way - and could therefore be necessary for implementing AI-based scenarios. If the outcome of the research task 1.3.1 highlights the necessity to specify/expose an MCP-Server to interact seamlessly with the NSAK framework, the implementation is a (must), otherwise a (could).

- A MCP-Server is specified and implemented
- Documentation how to connect to the MCP is provided.
- A form of authentication is provided.

Web GUI (could)

The goal is to design and implement a web-based graphical user interface that allows users to interact with the NSAK framework in a user-friendly way. The Web GUI will enable management of scenarios and drills, triggering scenario execution, and visualization of execution results and system status.

Success criteria:

- A simple Web GUI that interacts with the REST API has been implemented.
- The Web GUI displays a list of available scenarios and their states.
- The scenario's results can be viewed on the Web GUI
- Scenarios can be configured and executed via the Web GUI
- Authentication and authorization are explicitly out of scope.

1.3.3. Scenarios

AI – Purple Team (must)

The goal is to implement artificial intelligence according to the result of 1.3.1 and to explore the use of AI to support red, blue, and purple team exercises in the NSAK framework. The AI will be evaluated for its potential to assist in executing attacks in a scenario and analyzing detection results. The findings will help assess how AI could enhance and coordinate red and blue team activities across a network via the NSAK device.

Success criteria:

- The AI is integrated into NSAK as a scenario or as part of the framework, depending on the result of 1.3.1.
- The AI can execute CLI or Python tools to directly or indirectly interact with the attached network.
- An operator can interact with or stop the running AI.
- The result artifacts are stored and can be presented or reused for further scenarios.

Reproducible Test Environment (must)

The goal is to design and implement a reproducible and safe test environment for the NSAK framework that enables consistent execution and evaluation of scenarios and drills. The environment should provide a stable basis for testing and comparison of manual and AI-assisted scenarios.

Success criteria:

- The test environment can be set up reproducibly using a defined configuration or setup procedure with Infrastructure as code (e.g., Ansible)
- NSAK scenarios and drills can be executed consistently within the environment.
- The environment supports both manual and AI-assisted scenarios.
- Scenario results can be collected and analyzed for evaluation purposes.

Network Discovery – Red Team (must)

The goal is to implement a reference red team scenario for network discovery within the NSAK framework. The scenario should simulate reconnaissance activities to identify hosts, services, and network characteristics, and the results should be comparable to the AI-Agent approach.

Success criteria:

- A red team network discovery scenario is implemented in the NSAK framework.
- Discovered network information is collected as artifacts.
- The scenario can be executed reproducibly within the test environment.
- The results can be compared with those generated by the AI-assisted approach.

Intrusion Detection – Blue Team (should)

The goal is to define and implement a blue team reference scenario for intrusion detection within the NSAK framework. The scenario should allow observation and evaluation of suspicious network activity to assess detection capabilities in the test environment and compare the results with those of the AI approach.

Success criteria:

- A blue-team intrusion-detection scenario is implemented in the NSAK framework.
- Detection events or alerts can be observed and documented during scenario execution.
- Detection results can be compared with results from AI-assisted scenario execution.

1.3.4. Evaluation Goals

AI Scenario Evaluation (must)

The objective is to determine the effectiveness of the AI-based scenario introduced by the goal 1.3.3 within the network environment defined by the goal 1.3.3 The evaluation will assess whether the AI scenario successfully executed the expected red and blue team activities for the provided prompts and the quality of the results.

Success criteria:

- A methodical approach for the assessment is evaluated and documented.
- The results of the AI scenario for red and blue team activity are assessed.
- There is a conclusion on the effectiveness of the AI scenario for red and blue team activities.

AI vs Engineered Scenarios (must/should)

The goal of this evaluation is to compare the quality of AI-based red (network reconnaissance) (must) and blue (network intrusion detection) (should) team scenarios with that of manually engineered scenarios.

The comparison focuses on criteria such as execution reliability, reproducibility, and coverage. The results will help determine the potential benefits and limitations of AI-supported scenarios and provide insights into how AI could enhance future versions of the NSAK framework.

Success criteria:

- A methodical approach for the comparison is evaluated and documented.
- The results of the AI scenario are compared against the respective results of the red team scenario (must)
- The results of the AI scenario are compared against the respective results of the blue team scenario (should)
- There is a conclusion on the effectiveness of the AI scenario compared to manually engineered scenarios.

1.3.5. Out of Scope / Optional Feature

At the project management meeting on 05.03.26 (see Table **??** in Appendix A.7), the initial project scope was discussed. During planning, the team proposed using AI-based scenarios and comparing them with the implemented versions, which required reassessing certain milestones.

Running AI-based scenarios does not require a REST API or GUI. A reproducible test setup and further research are needed to properly evaluate this approach.

2. Related Research

2.1. Framework Comparison

Previous research, such as 'SOK: A Survey of Open-Source Threat Emulators', compares multiple types of threat emulation frameworks using both quantitative and qualitative metrics [1]. Among the evaluated frameworks, Atomic Red Team, Mitre Caldera, and Metasploit received high scores in categories as environments and operator, environment configuration and scenario execution.

n the following section, the selected frameworks are briefly introduced, and the unique selling point of the Network Swiss Army Knife (NSAK) is highlighted. As a point of reference, the cybersecurity community increasingly relies on the concepts of MITRE ATT&CK, which has established itself as a standard framework in the field since its introduction in 2014 [2, 3].

The MITRE ATT&CK is a globally accessible knowledge base of adversary tactics and techniques based on real-world observations [2]. The knowledge base is structured into TTP (Tactics, Techniques, and Procedures), which are organized into 14 attack behavior groups. The TTP is getting continuously updated and extended, which helps to defend against the capabilities of threat actors.

The evaluation of the proposed frameworks is based on information obtained from official documentation [4–6] and the findings of Zilberman et al. [1].

Characteristic	Metasploit Framework	Atomic Red Team	MITRE Caldera
Primary Purpose	Exploitation and penetration testing framework	Security testing through small atomic ATT&CK tests	Automated adversary emulation platform
Relation to MITRE ATT&CK	Partial mapping via modules	Direct mapping to ATT&CK techniques	Built around ATT&CK adversary emulation
Automation Level	Medium (scripts and modules)	Low (manual execution of tests)	High (automated attack chains)
Main Functionality	Exploit development, payload delivery, post-exploitation	Execute small ATT&CK technique tests	Simulate adversary campaigns across hosts
Architecture	Modular framework with exploits, payloads, and auxiliary modules	Collection of scripts/tests organized by ATT&CK techniques	Client-server platform
Typical Use Case	Penetration testing and vulnerability validation	Detection validation and security control testing	Red team automation and purple team exercises
Difficulty Level	Medium–High	Low–Medium	Medium–High

Table 2.1.: Comparison of Metasploit, Atomic Red Team, and MITRE Caldera ([1, 4–6])

Table formatting

As shown in Table 2.1, the key characteristics of the frameworks are differ greatly. **Atomic Red Team** provides libraries of attack scripts that implement individual Tactics, Techniques, and Procedures of MITRE ATT&CK. Each attack can be executed independently or chained together into a larger attack scenario. **Mitre Caldera** is a complex, agent based adversary simulation tool, whereas **Metasploit** is a framework that provides exploits, payloads and auxiliary modules. Further details of the frameworks are presented in the following sections. 2.1.12.1.22.1.3

2.1.1. Atomic Red Team

The Atomic Red Team repository provides documentation and configuration files for each technique. For every TTP, a YAML Ain't Markup Language (YAML) configuration file defines the execution parameters, while an enhanced Markdown file contains a human-readable description of the attack. This documentation includes a short description of the technique, execution instructions, and additional contextual information [5].

The framework can be executed directly from the command line and provides features such as logging, detailed error messages, and automatic prerequisite checks. If a technique cannot be executed, for example due to operating system constraints, the framework reports the issue and can optionally attempt to install the required prerequisites [5].

Zilberman et al. recommends deeper cybersecurity knowledge to use Atomic Red Team framework effectively [1]. The Open Source mentality, expandability and easily usable scripts can be a great asset for an independent framework such as the NSAK.

2.1.2. Caldera

Mitre Caldera is also built on the MITRE ATT&CK knowledge base and functions as a command-and-control center. A central server controls agents running on the target system, executes attacks called abilities, and sends the operation results back to the server. Every Ability is a single attack step built on the Tactics, Techniques, and Procedures. The framework ships with a graphical user interface and is primarily used for red team attack simulation, purple teaming, and detection engineering. The additional features, like a training plugin for educational purposes, include facts that can be configured as dynamic safe values, which can be stored and parsed in later steps [4].

With its provided interface and functionality, Caldera is most suitable as a threat emulator, offering a wide range of built-in features, including lateral movement and Command and Control (CNC) communication [1].

2.1.3. Metasploit

Metasploit is a widely used penetration testing framework for testing or attacking systems and exploiting known vulnerabilities. Metasploit follows a sequence of steps to identify a vulnerability in a target system, select a vulnerability for the attack, select an exploit that uses this vulnerability, and deliver a payload that runs on the target system. The configuration via the Command Line Interface (CLI) requires knowledge of the target network and system and requires partial configuration [6, 7].

According to Zilberman et al., an operator requires extensive cybersecurity knowledge to effectively use the framework. To support the operator, the GUI “Armitage”, which is not further evaluated in this work, can be used [1].

2.1.4. NSAK

The NSAK framework, as an independent universal Network Swiss Army Knife, is not confined and can leverage, if needed, different sets of attack frameworks. An approach could be to use an Artificial Intelligence (AI)-Enhanced Network Discovery Scenario, which provides the necessary information to configure a Metasploit attack that leads to an attack operation in Caldera and an atomic red team detection script. Hardware-based isolation and network position provide flexibility and should help reduce the operational costs of red and blue teams by reducing simple configuration overhead [8].

2.2. AI-Research

2.2.1. AI and Language Models

Language Model (LM) describes the technique for advancing language intelligence in machines and its development history can be divided into several stages. The first stage, Statistical Language Model (SLM) emerged around the 1990s, and the basic idea is to assume, that such a model can predict the next word. But the fact that an exponential number of transition probabilities are needed, caused a lack of accuracy and led limited success.

To overcome these limitations, Neural Language Model (NLM) was introduced to learn a distributed representation of words and capture more complex patterns, leading to a major breakthrough. Building on this, the advancement of pre-trained language models Pre-trained Language model (PLM), which are first trained on large datasets and then fine-tuned for specific tasks, represented another significant development.

Building on this development, Large Language Model (LLM)s further increases the scale of PLMs by expanding parameter count and computational power [9].

In large language models, parameters are the trainable numerical values that define how the model processes input and generates output. During training, these parameters are continually adjusted to enable the model to learn patterns and language context [10]. The models most of us use today, such as ChatGPT, Claude and Gemini are pre-trained LLMs used for language inference.

These advances led to MCP, a protocol that allows tools and services to expose an interface specifically for AI. Further, the introduction of AI-agents is enabling a new level of autonomy for achieving concrete goals with LLMs. The combination of these technologies is now resulting in systems, where LLMs empower autonomous agents capable of decision-making to execute complex tasks.

2.2.2. MCP

Anthropic, a US software company focused on AI research, introduced the protocol in 2024. The MCP was introduced as an open source standard that enables AI assistants to connect to external systems, including databases, business tools, and development environments [11]. Dynamic tool discovery and schema negotiation is helping clients and agents to access these functions in a standardized manner and allows autonomous interaction. Additionally, MCP allows information to flow in both directions, so the model can send requests, and tools can send updates or events.

This architectural approach shifts the AI integration from static toward an interoperable ecosystem of AI-enabled services. On the other hand, the flexible design and open source approach of MCP unlocks multiple novel and systematic security risks that are in need of enhanced security and governance [12].

2.2.3. Agentic-AI

Agentic Artificial Intelligence (Agentic-AI) can be understood as an advanced machine intelligence that is capable of perceiving, reasoning, and acting with a certain level of autonomy. Traditional systems and agents are more limited to executing predefined commands, whereas Agentic-AI is not restricted in this way. Due to their ability to adapt in real time, they can continuously refine their knowledge, techniques, and data. The ability to access multiple inputs, from natural language processing to sensor data, allows Artificial Intelligence Agent (AI Agent)s to develop a good understanding of their environment [13].

Multi Agent System (MAS) defines a way in which multiple specialist agent coordinate and cooperate with each other to solve a task, that would not be solvable to one agent alone.

Therefore, the agents need a form of interaction that allows them to act reactively, often through a shared mental model, which helps create a common understanding of the complex tasks to be solved. To implement a MAS efficiently, modern design principles such as encapsulation and modularity are important, which are helping to balance resource usage of the system and LLM [14]

Huang et al. provide evidence that Agentic-AI and MAS are capable of solving complex tasks across various sectors, including security and banking. However, these systems also introduce new potential vulnerabilities, both within AI Agent systems themselves and through their interactions with external environments.

The identified vulnerabilities can be categorized into two types: accidental and deliberate. Accidental vulnerabilities include software bugs, logical errors introduced by agents, hardware malfunctions, and poor data quality. Deliberate

attacks, on the other hand, encompass adversarial attacks targeting LLMs, data poisoning, and model theft.

To mitigate these risks, a comprehensive framework consisting of governance mechanisms, proactive monitoring, regulatory compliance, and rigorous testing is required to ensure the secure and reliable operation of agentic systems [15].

Fazit MCP and Agentic AI

In the cybersecurity podcast Risky Business, it is mentioned that MCP is "dead". The open-source strategy of Anthropic (2.2.2) has led to the development of multiple MCP implementations that were not designed with a strong focus on security, but rather on rapid adoption of a novel technique.

This provocative claim is based on the observation that, in order to handle the excessive MCP tooling, AI Agents may instead rely on direct access to a root shell, as it provides a more efficient way of interaction [16].

How and why is this relevant for our thesis? how could we implement it? To which findings should we pay attention? What can we ignore for now?

Evaluated Frameworks and MCP

Caldera uses a MCP plugin as a bridge between the model and the Representational State Transfer Application Programming Interface (REST-API), allowing the agent to plan and issue commands in a manner similar to a human operator.

Caldera introduces three main research areas. Firstly, a planner determines the order and selection of abilities and can dynamically construct new adversaries based on user requirements through prompt-based interaction.

Secondly, an adversary factory model is used to generate adversaries tailored to the operator's specific use cases.

Thirdly, a forward-deployed agent is enhanced with Retrieval Augmented Generation (RAG) for Cyber Threat Intelligence (CTI), leveraging Structured Threat Information Expression (STIX)-based data to create reproducible and testable adversaries [17].

The MCP-Server, provided by **Metasploit**, enables a LLM to interact with the Metasploit Framework dynamically. The Interface covers module information (a Metasploit module is a known exploit), exploitation workflows, payload generation, session management and handler management to help manage an attack. The documentation includes a security warning that emphasizes the importance of reviewing every prompt, using a test environment, and being aware of post-exploitation commands. **Metasploit** is a powerful exploitation framework, and the simplicity that AI could bring can also harm. [18]

The **Atomic Red Team** MCP Server provides an interface that allows AI systems to access and use tests from Atomic Red Team. It enables searching, validating, and optionally executing attack techniques (e.g., based on MITRE ATT&CK). [19]

What about the other frameworks? What about AI Agents? What can we learn from their approach? How does it affect our strategy?

Outcome Comparison

The framework comparison highlights the difference between the NSAK and the established framework. Each compared framework performs a specific task in a complex operation, and the NSAK device can leverage the strengths of multiple aspects of a different set of tools. Building upon these insights, the NSAK is designed to extend and integrate these capabilities into a unified and adaptable system and the ai pivot gets further elaborated in this thesis

3. Concepts

3.1. Framework Resources

The NSAK framework is built around four resource types that were originally defined in "Project 2" 1.2 [8]. Each resource type has a distinct role in the framework and is stored as a YAML file in the resource library, from which the framework loads and manages them at runtime.

3.1.1. Device

A **device** is a physical or virtual machine that is capable of running the NSAK framework. It serves as the execution host for scenarios and drills and is typically deployed at a strategic network position — for instance, between segments or inline on a link — to give it the access required for the operations it performs.

A device exposes its network configuration to the framework by declaring its interfaces and their Internet Protocol addresses in its resource YAML. This allows scenarios and drills to resolve interface names and target addresses without hard-coding network details, making the same scenario portable across different devices and network environments.

3.1.2. Environment

An **environment** represents a specific network topology, including its infrastructure components, hosts, and services [8]. It describes the target network context in which a scenario is intended to operate — for example, a simple client–server setup over a switch, a Wireless LAN access point with connected clients, or a segmented business network.

Environments are referenced by scenarios to make explicit which network context they are designed for, helping operators select and validate the right configuration before executing an operation.

3.1.3. Drill

A **drill** is a self-contained, reusable unit of execution with a specific, well-defined goal [8]. Its goal can be an active or passive attack step, network discovery, traffic capture, a configuration change, or any other discrete action relevant to a red or blue team activity.

Examples of drills include Address Resolution Protocol spoofing, host discovery via Address Resolution Protocol scan, transparent Transmission Control Protocol proxying, traffic capture with T-Shark, and network configuration tasks such as enabling IP forwarding or Network Address Translation (NAT).

Each drill declares the system and Python packages it requires as well as a typed `interface` that lists its input arguments and return type. The return value of one drill can be consumed as the input of a subsequent drill within a scenario, enabling data to flow through a chain of execution steps.

3.1.4. Scenario

A **scenario** orchestrates a sequence of drills targeted at one or more environments and describes a concrete red or blue team use case [8]. Where a drill is a single reusable action, a scenario gives that action meaning by placing it in context: it defines which drills are executed, in which order, and against which environment.

Like drills, scenarios declare a typed `interface` with input arguments, allowing operators to parameterise a scenario at execution time via the CLI without modifying the scenario definition itself. This separation keeps scenarios reusable across different network configurations while still allowing runtime flexibility.

3.2. Configuration Management

The NSAK framework distinguishes three configuration layers, each with a different scope, lifetime, and owner. Keeping them separate avoids coupling concerns that change at different rates and makes it easier to reason about where a given piece of configuration comes from.

3.2.1. Static Configuration

The static configuration layer resolves deployment-specific values that are fixed for the entire lifetime of a process. It establishes the filesystem layout that all

other parts of the framework depend on: where the resource library is located, where runtime data is written, and what the base directory of the installation is.

Because these values derive from a single base path, the whole framework can be relocated — into a Docker container, a test fixture, or a CI pipeline — by changing one environment variable. No other code needs to be aware of the deployment context.

3.2.2. Runtime Configuration

The runtime configuration layer holds mutable state that must be shared across separate CLI invocations. Each invocation of the CLI is a new process, so any state that needs to carry over — such as which device is currently active — must be persisted between calls. The runtime configuration is therefore stored as a YAML file and loaded at the start of every invocation. Commands that change state write the updated configuration back to disk before they exit. This design keeps the state model simple: there is one authoritative file on disk, and any process can read or update it without requiring a background service. Loading is deferred until the configuration is first accessed rather than at import time. This avoids unnecessary disk reads when the configuration is not needed and keeps the startup fast. If no configuration file exists yet, a default is created automatically on first access, so no explicit initialization step is required from the user.

3.2.3. Resource Configuration

The resource configuration layer is owned by the resource library rather than the framework core. Each resource — device, drill, scenario, or environment — declares its own configuration inside a YAML file stored alongside its implementation. The framework reads these files to discover and load resources at runtime, without requiring any code changes when adding new resources.

Mergeable YAML Structure

All resource YAML files follow a common structure in which resources are nested under a resource-type key (e.g. `drills:`, `scenarios:`). This structure ensures that YAML files from different parts of the library can be merged into a single in-memory data structure without key conflicts, which is a prerequisite for supporting multiple library paths and composite library configurations in the future.

Typed Interface Arguments

Drills and scenarios declare a typed `interface` in their YAML file that lists the arguments they accept, each with an explicit type annotation and an optional default value. The framework reads these annotations at runtime and automatically generates the correct CLI options for each resource. This means that adding a new argument to a drill or scenario immediately exposes it as a typed CLI option without requiring any changes to the framework or the CLI code. Separating the argument declaration from the CLI implementation keeps the two concerns independent and makes each resource's interface self-describing.

4. Methodology

5. Implementation

5.1. Configuration Management Implementation

The configuration management in NSAK is organised into three distinct layers, each serving a different scope and lifetime.

The **static configuration** layer resolves fixed filesystem paths at import time. It derives all relevant paths from a single base directory, which can be overridden via an environment variable to support different deployment contexts such as Docker containers or automated test environments.

The **runtime configuration** layer holds mutable state that persists across CLI invocations. It is loaded from a YAML file on every command execution and written back whenever state changes, for example, when a different device is loaded. This layer serves as the single source of truth for any information that must be shared between separate CLI calls.

The **resource configuration** layer is defined by the resource library itself. Each resource type — devices, drills, scenarios, and environments — declares its configuration inside a YAML file stored in the library. For devices, this includes network interface assignments, while drills and scenarios additionally declare their typed argument interfaces, which the CLI uses to generate the correct command options at runtime.

5.1.1. NSAK Configuration Management

The NSAK configuration is split into two distinct layers: a static layer that resolves fixed paths at import time, and a mutable runtime layer that is persisted to disk and loaded on every invocation.

Static Configuration

The static configuration is defined in `src/nsak/core/settings.py` and shown in listing 1. It derives all relevant paths from a single `BASE\_PATH`, which is either supplied via the `NSAK\_BASE\_PATH` environment variable or resolved automatically to the repository root at import time. `RUN\_PATH` points to the `run/` directory where runtime data such as the persisted config is stored, and `LIBRARY\_PATHS` holds the set of directories that are searched for resource libraries. Overriding `NSAK\_BASE\_PATH` is the primary mechanism to run NSAK from a different base directory, e.g. inside a Docker container or during automated testing.

```python
import os
from pathlib import Path

from dotenv import load_dotenv

load_dotenv()


def get_base_path() -> Path:
    """
    Returns the default base path.

    :return:
    """
    base_path = os.getenv("NSAK_BASE_PATH", None)

    if base_path is not None:
        return Path(base_path)

    return Path(__file__).resolve().parents[3]


BASE_PATH = get_base_path()
RUN_PATH = BASE_PATH / "run"
LIBRARY_PATHS = {BASE_PATH / "lib"}
DOCKER_CONTEXT = BASE_PATH
OLLAMA_BASE_URL = os.getenv("NSAK_OLLAMA_BASE_URL", None)
```

Listing 1: NSAK Static Configuration

Runtime Configuration

The runtime configuration is defined as the `Config` dataclass in `src/nsak/core/\_config.py` and holds all state that can change during normal operation, currently the `debug` flag and the active `Device`. It is persisted as a YAML file at `run/config.yaml` and loaded on every CLI invocation.

To avoid loading the configuration on import, the module-level `config` object is wrapped in a `lazy\_object\_proxy.Proxy`. The proxy defers the actual `Config.load()` call until the object is first accessed. Since the root CLI group imports `config` on every command invocation (listing 3), this ensures the configuration is always initialised before any command logic runs. If `run/config.yaml` does not yet ex-

ist, `Config.init()` is called automatically to create a default configuration and write it to disk, eliminating the need for manual setup on first run.

```
import dataclasses
import logging
from dataclasses import asdict
from ipaddress import IPv4Interface, IPv6Interface
from pathlib import PosixPath
from typing import Any, Self

import yaml
from lazy_object_proxy import Proxy
from yaml import SafeDumper, ScalarNode

from .device import Device
from .settings import RUN_PATH


def represent_as_string(dumper: SafeDumper, data: Any) -> ScalarNode:  # noqa: ANN401
    """
    Used for serializing objects to strings (e.g., IPInterface or PosixPath).

    :param dumper:
    :param data:
    :return:
    """
    return dumper.represent_str(str(data))


yaml.add_representer(IPv6Interface, represent_as_string, Dumper=SafeDumper)
yaml.add_representer(IPv4Interface, represent_as_string, Dumper=SafeDumper)
yaml.add_representer(PosixPath, represent_as_string, Dumper=SafeDumper)


@dataclasses.dataclass(kw_only=True)
class Config:
    """
    Class for loading and saving the configuration.
    """

    debug: bool
    device: Device

                                25

    @classmethod
    def init(cls) -> Self:
        """
        Initialize the default config.
        """
        from .device.device_manager import DeviceManager

        _config = cls(
            debug=True
```

```python
import click

from .device import device_group
from .drill import drill_group
from .environment import environment_group
from .scenario import scenario_group


@click.group()
def cli() -> None:
    """
    CLI root.
    """
    # Load the configuration for initialization
    from nsak.core import config  # noqa


cli.add_command(scenario_group)
cli.add_command(drill_group)
cli.add_command(device_group)
cli.add_command(environment_group)
```

Listing 3: NSAK CLI Initialisation

Mutations to the runtime configuration always follow the same pattern: a manager method (e.g. `DeviceManager.load()`) updates the relevant field on the `config` object and calls `config.save()` to persist the change. Custom YAML representers are registered for `IPInterface` and `PosixPath` so that these types are serialised as plain strings rather than Python-specific tags. Listing 4 shows an example of the persisted `run/config.yaml` after loading the `bananapi\_r4` device.

```yaml
debug: true
device:
  id: bananapi_r4
  name: BananaPI R4
  path: <base-path>/lib/devices/bananapi_r4
  author: vonal3@bfh.ch
  repository: <repository-url>
  configuration:
    network:
      ethernets:
        lan1@eth0:
          addresses:
            10.10.10.30/24:
              is_target: true
              is_management: false
```

Listing 4: Persisted Runtime Configuration (run/config.yaml)

5.1.2. Device Configuration Management

In the context of "Project 2"1.2, this resource type was already defined conceptually, and the BananaPI R4 was added as an example in the library under `lib/devices/bananapi_r`
But the resource type was not actually added to the NSAK core, and to manage device configurations, we need to implement the Device resource type first.

Create New Device Resource Type

Similar to the other resource types, a YAML specification for defining devices in the library and multiple classes for representing and managing devices in the core are required. Because the existing resource YAML specifications do not allow merging YAML files, this implementation proposes a new base structure that fulfills this trait. Section 5.1.2 describes the necessary changes to the existing YAML specifications. The listing 5 shows the initial specification for a device resource, which will get extended later on with a section for configuration management.

```yaml
devices:
    <str:resource-id>:
        id: <str:device-id>
        name: <str:device-name>
        description: <str:device-description>
        author: <str:author-email>
        repository: <str:resource-repository>
```

Listing 5: Device Resource YAML Specification

In the table 5.1 below, the required Python classes are listed and described. The classes will be located under `src/nsak/core/device` within their respective files.

Class	File Name	Description
Device	device.py	A dataclass, which represents a device resource as objects during runtime (usually quite similar to the yaml specification).
DeviceLoader	device_loader.py	This class returns Device objects and is used by the DeviceManager to find and load device resources from the library.
DeviceManager	device_manager.py	The manager class implements the business logic and exposes an API, which can be used by the CLI.

Table 5.1.: Device Resource Classes

In “Project 2”1.2, it became already clear that the described classes share a lot of boilerplate code between each other. For this reason, the base classes described in 5.1 were added to eliminate duplicated logic, thereby improving compliance with the Don’t Repeat Yourself principles. The classes will be located under `src/nsak/core/resource` within their respective files and the adoption of the existing classes is described in section 5.1.2.

To complete the integration of the device resource, the new functionality must be exposed via CLI. The available bash commands are shown in the following listing 6.

Class	File Name	Description
Resource	resource.py	Base class for all resources: Scenario, Drill, Environment and Device.
ResourceLoader	resource_loader.py	Base class which handles searching and loading resources from the filesystem.
ResourceManager	resource_manager.py	Base class which defines common methods for all resource manager classes.

Table 5.2.: Resource Python Classes

```
# Show usage, options and subcommands for the device resource
nsak device --list

# List all devices found in the resource library
nsak device list

# Output:
❯ nsak device list
ID           NAME         PATH
bananapi_r4  BananaPI R4  <repository>/lib/devices/bananapi_r4
```

Listing 6: NSAK CLI: List Devices

Device Configuration Management Implementation

To allow a device to expose its configuration options, the YAML specification needs to be extended accordingly. For now, the focus is only on network configuration, as this is the most important information for the scenarios and drills. The specification was designed so that other sections can be added later. The following excerpt 7 shows the specification for the configuration section of the device resource YAML.

```yaml
devices:
    <str:resource-id>:
        # ...
        configuration:
          network:
            ethernets:
              <str:intreface-name>:
                addresses:
                  <str:cidr>:
                    is_target: <boolean>
                    is_management: <boolean>
```

Listing 7: Device Resource Configuration YAML Specification

The following code snippet, located under `src/nsak/core/device/device_loader.py` 8 shows how the YAML configuration gets parsed to the respective datastructures defined in `src/nsak/core/network/configuration.py` and `src/nsak/core/device/device.p`

```
class DeviceLoader(ResourceLoader[Device]):
    """
    Finds and loads devices from the library paths.
    """

    ResourceClass = Device

    @classmethod
    def _parse_device_configuration(
        cls, data: dict[str, Any]
    ) -> DeviceConfigration | None:
        if "configuration" not in data:
            return None
        network = data["configuration"].get("network") or {}
        if network == "auto":
            ethernets = {}
            for interface in get_network_interfaces():
                ethernets[interface.name] = {
                    "addresses": {
                        ip: {
                            "is_target": True,
                            "is_management": False,
                        }
                        for ips in interface.ips.values()
                        for ip in ips
                    }
                }
        else:
            ethernets = network.get("ethernets") or {}

        return DeviceConfigration(
            raw=data["configuration"],
            network=NetworkConfiguration(
                ethernets={
                    interface: EthernetConfiguration(
                        name=interface,
                        addresses={
                            ip: IPConfiguration(
                                ip=ip_interface(ip),
                                is_target=address.get("is_target",
                                False),
                                is_management=address.get("is_management",
                                False),
                            )
                            for ip, address in
                            ethernet.get("addresses",
                            {}).items()
                        },
                    )
                    for interface, ethernet in ethernets.items()
                }
            )
```

```python
@dataclasses.dataclass(frozen=True, kw_only=True)
class IPConfiguration:
    """
    Represents an ip configuration on an interface.
    """

    ip: IPInterface
    is_target: bool
    is_management: bool


@dataclasses.dataclass(frozen=True, kw_only=True)
class EthernetConfiguration:
    """
    Represents an ethernet node.
    """

    name: str
    addresses: dict[str, IPConfiguration]
```

```python
@dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
class DeviceConfigration:
    """
    Represents the device configuration.
    """

    raw: object
    network: NetworkConfiguration


@dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
class Device(Resource):
    """
    Represents a device.
    """

    NAME = "device"
    KEY = "devices"

    id: str
    name: str
    description: str
    path: Path
    author: str
    repository: str
    configuration: DeviceConfigration | None = None
```

Listing 9: Device Configuration Datastructures

The following listing shows the newly available CLI commands for managing device configurations 10:

```
# List all subcommands for the Device resource
nsak device --help

# List all available devices
nsak device list

# Show the device details and configuration
nsak device show <str:device-id>

# Load a device into in the NSAK configuration
nsak device load <str:device-id>

# Show the currently loaded device stored in the NSAK
↪  configuration
nsak device loaded

# Reset the loaded device
nsak device unload
```

Listing 10: Device Configuration CLI Management

Refactor Existing Resources

The implementation of the device resource 5.1.2 introduced an improved base structure for the resource YAML specifications and three base classes which are abstracting the common structure and functionality. This section describes the resulting changes in the existing code and resource YAML specifications.

As already stated in section 5.1.2 all resource type specific classes were refactored to inherit from their respective base classes as described in table 5.1. With the example of the `EnvironmentLoader`, the diff below 5.1 illustrates how the abstraction of common properties and methods allows for the removal of a lot of boilerplate code.

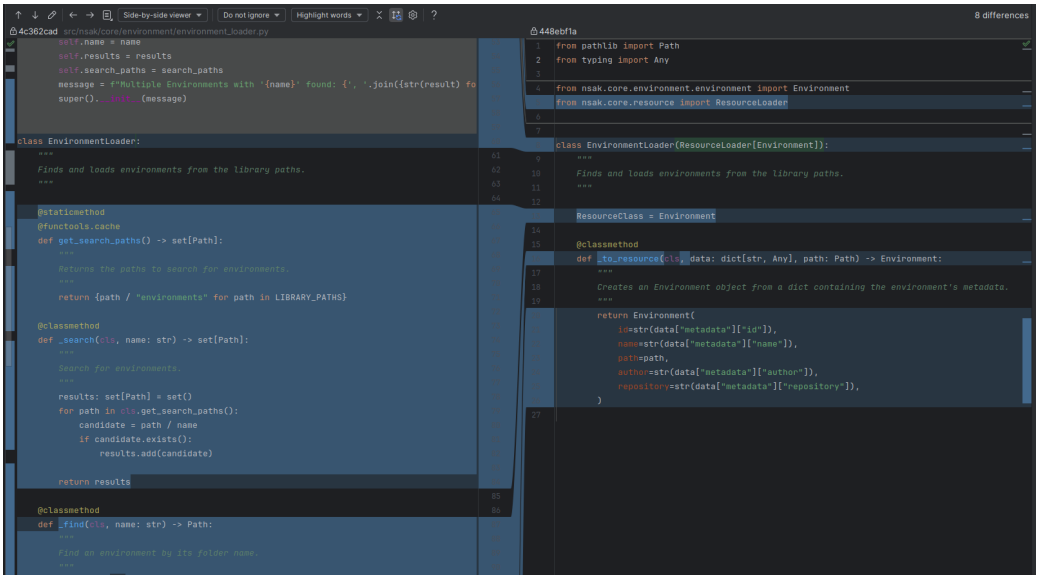

Figure 5.1.: EnvironmentLoader Base Class Refactoring Diff

The structure of all resources was aligned by nesting the resource under their respective resource type key (`scenarios`, `drills`, `environments`, and `devices`), so that we could merge all YAML resource files into a single library data structure without key conflicts. In the YAML specification 11 below, the new base structure derived from the device resource 5.1.2 is illustrated. All existing resource YAML files in the library were adjusted accordingly, and thanks to the introduction of the `ResourceLoader` base class, the required adjustments in the code were minimal.

```yaml
# Resource type version was removed, as it was not used anyway

# scenarios|drills|environments|devices
<str:resource-type-key>:
    # unique resource identifier
    <str:resource-id>:
        # The "metadata" key is removed and the children are now
        ↪  nested directly under the resource
        name: <str:name>
        description: <str:description>
        author: <str:email>
        repository: <str:link>
        # ... resource type specific properties, which
        ↪  previously were at the top level
```

Listing 11: Mergeable Resource YAML Specification

5.1.3. Scenario & Drill Configuration Management

Both the scenario and drill resource types were already fully implemented in "Project 2"1.2. Their YAML specifications already contained an `interface` section describing argument and return types, but this information was purely documentary and had no effect at runtime.

As part of this project, the `interface.arguments` section was extended 12 so that each argument carries an explicit `type` annotation and an optional `default` value.

```yaml
<str:resource-type-key>:
    <str:resource-id>:
        # Unchanged properties...

        interface:
            arguments:
                <str:argument-name>:
                    type: <str:type-annotation>
                    default: <any:default-value>
            return_type: <str:type-annotation>
```

Listing 12: Updated Interface Specification

The CLI reads these annotations at runtime to generate typed command options

for each drill and scenario automatically, without requiring any additional hand-written code. The `discover\_hosts` drill in listing 13 illustrates this: its single `interface` argument of type `str` is translated directly into a `--interface` option on the generated CLI command.

```yaml
drills:
  discover_hosts:
    name: Discover Hosts
    author: vonal3@bfh.ch
    repository:
     ↪  https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffe

    dependencies:
      system:
        - arp-scan
      python: [ ]

    interface:
      arguments:
        interface:
          type: str
      return_type: NetworkDiscoveryResultMap
```

Listing 13: discover_hosts Drill YAML

Scenarios follow the same `interface` convention and additionally declare the drills and environments they depend on. The `mitm` scenario in listing 14 chains the `discover\_hosts`, `arp\_spoof`, and `transparent\_tcp\_proxy` drills together and exposes a single typed `interface` argument, which the CLI surfaces in the same way as for individual drills.

```yaml
scenarios:
  mitm:
    id: mitm
    name: MITM Scenario
    author: vonal3@bfh.ch
    repository:
     ↪  https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffe

    drills:
      - discover_hosts
      - arp_spoof
      - transparent_tcp_proxy

    environments:
      - simple_tcp_client_server

    dependencies:
      system:
        - arp-scan
        - dsniff
        - iptables
      python:
        - scapy

    interface:
      arguments:
        interface:
          type: str
      return_type: None
```

Listing 14: mitm Scenario YAML

6. Evaluation

7. Discussion

8. Conclusion

Bibliography

[1] Polina Zilberman, Rami Puzis, Sunders Bruskin, Shai Shwarz, and Yuval Elovici. Sok: A survey of open-source threat emulators. arXiv preprint, 2020.

[2] Blake E. Strom, Andy Applebaum, Doug Miller, Adam Nickels, Cody Pennington, and Chris Thomas. Mitre att&ck: Design and philosophy. Technical report, MITRE Corporation, 2020.

[3] Bader Al-Sada, Alireza Sadighian, and Gabriele Oligeri. Mitre att&ck: State of the art and way forward. *ACM Computing Surveys*, 2023.

[4] MITRE Corporation. Mitre caldera documentation, 2024.

[5] Red Canary. Atomic red team, 2024.

[6] Rapid7. Metasploit framework documentation, 2024.

[7] David Kennedy, Jim O'Gorman, Devon Kearns, and Mati Aharoni. *Metasploit: The Penetration Tester's Guide*. No Starch Press, 2011.

[8] Frank Gauss and Lukas von Allmen. Nsak (network swiss army knife). https://github.com/nsak-team/nsak, 2026. Accessed: 2026-01-10.

[9] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2):1–124, 2023.

[10] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.

[11] Anthropic. Model context protocol. https://www.anthropic.com/news/model-context-protocol, 2024. Accessed: 2026-03-18.

[12] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers, 2025.

[13] K Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[14] J Huang K Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[15] K Jackson C Hughes J Huang, K Huang. Ai agent safety and security considerations. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[16] Risky Business Feature Podcast. ...mcp is dead. https://risky.biz/, 2026. Podcast episode , Accessed: 2026-03-18.

[17] Ethan Michalak and Mark Perry. Connecting llms to caldera with the mcp plugin. https://medium.com/@mitrecaldera/connecting-llms-to-caldera-with-the-mcp-plugin-da1450254827, 2025. MITRE Caldera Blog, Accessed: 2026-03-16.

[18] GH05TCREW. Metasploit mcp server. https://github.com/GH05TCREW/MetasploitMCP, 2025. Accessed: 2026-03-16.

[19] Hare Sudhan. Atomic red team mcp server. https://github.com/cyberbuff/atomic-red-team-mcp, 2025. Accessed: 2026-03-16.

[20] Advisera. Iso 27001 risk assessment, treatment, and management: The complete guide, 2025. Accessed: 2026-03-04.

[21] Simplr. General recommended vram guidelines for llms, 2025. Online; accessed 10 March 2026.

List of Figures

List of Tables

List of Listings

Glossary

This document is incomplete. The external file associated with the glossary 'main' (which should be called `thesis.gls`) hasn't been created.

Check the contents of the file `thesis.glo`. If it's empty, that means you haven't indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can't be generated. If the file isn't empty, the document build process hasn't been completed.

If you don't want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

  ```
  \usepackage[automake]{glossaries-extra}
  ```

- Run the external (Lua) application:

  ```
  makeglossaries-lite.lua "thesis"
  ```

- Run the external (Perl) application:

  ```
  makeglossaries "thesis"
  ```

Then rerun LaTeX on this document.

This message will be removed once the problem has been fixed.

A. Project Management

A.1. Stakeholders

Hj. Wenger: Instructor and Network Specialist
Urs Keller: Expert and Entrepreneur

A.2. Agile Management Process

The project followed an interactive development approach using Gitlab for issue tracking and planning. We defined our project management in a Scrum-like manner, but we individualized the process according to the projects scope and our needs. Work items where organized into epics, milestones and issues. Each iteration lasts two weeks. We decided to hold the following Scrum ceremonies after each sprint:

Sprint Planning	**Sprint Review**	**Sprint Retro**	**Checkpoint**
Select milestone with sprint goal	Close open tickets	Reflect on sprint	Present sprint results
Check issues for DoR and obstacles	Gather team feedback	Identify improvements	Get stakeholder feedback
Ensure issues are clear	Validate delivered features	Discuss what went well	Define next tasks
Create sprint backlog	Review progress	Discuss issues/problems	Discuss issues/problems

Table A.1.: Key Scrum Meetings

The workflow used for ticket tracking consisted of the following stages:

A.2.1. Issue Lifecycle:

Buckets	Description
Backlog	List of all tasks and features planned for the project.
Todo	Tasks selected for the current sprint that are ready to be worked on.
Q&A	Tasks waiting for clarification, review, or testing.
Done	Completed tasks that meet the defined acceptance criteria.

Table A.2.: Issue Board

A.2.2. Definition of Ready (DoR)

Category	Criteria
Clarity	▶ Clear and precise title ▶ Objective or problem described in two to three sentences ▶ Scope clearly defined
Scope	▶ Affected module specified (drill, scenario, core, container, frontend) ▶ External dependencies identified
Validation	▶ Testable acceptance criteria defined ▶ Expected behavior clearly specified

Table A.3.: Definition of Ready Criteria for Tickets

A.2.3. Definition of Done (DoD)

Category	Criteria
Implementation	▶ All acceptance criteria fulfilled ▶ Container execution verified ▶ Pre-commit hooks pass ▶ Logging and error handling consistent
Testing	▶ Feature or bug fix tested ▶ Acceptance criteria verifiable (automated or manual) ▶ Relevant edge cases considered ▶ Execution reproducible
Documentation	▶ README updated if required ▶ Architecture changes documented ▶ Thesis relevance briefly noted ▶ Limitations documented ▶ Required documentation stored for thesis or appendix

Table A.4.: Definition of Done Criteria for Tickets

A.3. Project Structure

This section describes the projects structure and list the goals as milestones linked to their respective page in GitLab.

A.3.1. Milestones

A milestone represent crucial steps toward the overall project goal. Per our definition a milestone has at least one epic and the mandatory milestones are listed in TableA.5

GitLab Milestones
Project Management
Research AI Integration
Research: Compare Frameworks
NSAK Framework: Configuration Management
NSAK Scenario: Network Discovery - Red Team
NSAK Scenario: AI - Purple Team
Reproducible Test Environment

Table A.5.: Project Milestone Overview in GitLab

Project Planning

19.02.2026 – 10.03.2026

This is the initial milestone which is used to plan the whole project and bachelor thesis.

- ✓ The bachelor thesis is planned as an agile (scrum-like) project
- ✓ A risk analysis for the project was conducted and the identified risks are assessed and addressed
- ✓ All meetings, deadlines, scrum ceremonies and sprints are added to the Outlook calendar
- ✓ The goals of the thesis are defined and categorized as “must”, “should” and “could”
- ✓ The goals are created as milestones and divided into epics, categorized by the labels: Project Management, Research, Framework, Implementations, Documentation
- ✓ We know how we coordinate the project with the stakeholders (instructor and expert)

Research: AI Integration

10.03.2026 – 22.03.2026

The goal of this research milestone was to investigate how AI can support automated security testing and adversary emulation. The research evaluated potential integration points where AI could enhance scenario execution, automation,

decision-making, and analysis capabilities of the NSAK framework. The milestone was extended by three days following input from the expert and instructor to clarify the intended interaction between LLMs and NSAK.

- ☐ How is AI currently used in cybersecurity automation?
- ☐ Which tasks in adversary emulation can be supported or automated by AI?
- ☐ How could AI interact with the NSAK scenarios and drills (e.g. Agents, MCP)?
- ☐ What architectural changes would be required to integrate AI into NSAK?

Research: Compare Frameworks

10.03.2026 – 19.03.2026

This milestone focused on the research and comparison of existing security automation frameworks that could potentially support or influence the NSAK scenario and drill execution model. The evaluation included frameworks such as MITRE CALDERA and Atomic Red Team, examining their execution models, configuration and parameter handling, extensibility mechanisms, and the structure of reusable test definitions.

The expected outcomes were:

- ☐ A short comparison of selected frameworks
- ☐ A summary of relevant architectural concepts
- ☐ An initial assessment of how these frameworks relate to NSAK

NSAK Framework: Configuration Management

10.03.2026 – 19.03.2026

This milestone defined the configuration model and execution framework for NSAK scenarios. It introduced a structured approach for configuring scenarios and drills, including typed argument declarations, and the generation of a resolved run configuration. The goal was to provide a flexible and reproducible configuration system that allows scenarios and drills to expose parameters and execute in a defined order.

- ✓ Drills and scenarios can expose runtime parameters as configuration objects
- ✓ The configuration objects can be set via CLI arguments
- ✓ A configuration management system is implemented in the NSAK framework

NSAK Scenario: Network Discovery – Red Team

02.04.2026 – 16.04.2026

Network discovery is one of the most essential Red Team activities in the reconnaissance phase. This milestone covered the implementation of a scenario that identifies network participants and services, producing a structured network map that can be consumed by subsequent drills such as Address Resolution Protocol spoofing.

Goals (Must):

- ☐ The scenario identifies the available ports and interfaces on the device
- ☐ The scenario discovers the available subnets and obtains valid addresses
- ☐ The scenario discovers addresses, addresses and services (port scan)
- ☐ The scenario supports at least a fast full-search mode
- ☐ The scenario returns a network map as a structured data type

Goals (Could):

- ☐ A human interaction hook allows the operator to select the discovery mode
- ☐ Additional modes: slow full search, distinct search

NSAK Scenario: AI – Purple Team

19.03.2026 – 16.04.2026

This milestone explored what happens when an AI agent is given full access to the NSAK framework for both Red and Blue Team activities. The scenario is intentionally open-ended, reflecting the research nature of the milestone.

Goals (Must):

- ☐ The operator can provide a custom prompt or select a predefined Red or Blue Team prompt
- ☐ The scenario runs an AI agent with a remote connection
- ☐ An operator can interact with the agent via CLI at all times
- ☐ The scenario logs all relevant actions and network traffic locally
- ☐ The AI can trigger an e-mail alert to notify an operator
- ☐ The scenario has a kill switch to stop execution at any time

Goals (Should):

- ☐ Human interaction hook via Web GUI
- ☐ Remote logging and sniffing
- ☐ Kill switch with full device shutoff

Reproducible Test Environment

19.03.2026 – 02.04.2026

The goal of this milestone was to design and implement a reproducible and safe test environment for the NSAK framework, providing a stable basis for consistent execution and evaluation of both manual and AI-assisted scenarios.

- ☐ The test environment can be set up reproducibly using Infrastructure as Code (e.g. Ansible)
- ☐ NSAK scenarios and drills can be executed consistently within the environment
- ☐ The environment supports both manual and AI-assisted scenarios
- ☐ Scenario results can be collected and analyzed for evaluation purposes

A.3.2. Epics

consist of multiple stories and contribute to fulfilling milestones.

A.3.3. Issues

Each Story is assigned a weight, sprint, milestone and epic and are usually represented by an “Issue” in gitlab.

A.4. Sprints / Iterations

A.4.1. Planning

“Task” and “Issue”, the terms are often used interchangeably with “Stories”. Stories must meet the A.2.2 before implementation and A.2.3 before closing.

A.5. Planning and Estimation

The following estimation strategy was used:

- 1 hour corresponds roughly to 1 weight point
- Iteration length: 2 weeks
- Weights per iteration: 40h per person = 80 weight per iteration
- Sprint planning duration: 1.5 hours per iteration
- Sprint review duration: 1 h per iteration
- Sprint retro duration: 1 h per iteration

A.6. Project Roadmap

To keep track of the overall progress in the project, the project team uses the "Roadmap" feature in GitLab. After each sprint planning meeting we create a new screenshot of the roadmap and add it to the according sprint section in A.7. The first sprint planning is an exception, because at this point the diagram would have been empty.

A.7. Sprint Ceremonies

A.7.1. Sprint 1: 19.02 –10.03.2026

Sprint Planning 1: 19.02.2026

For the planning of sprint 1 we derived all issues from the milestone project management A.5 and had an overall weight from 89.

Sprint Review 1: 10.03.2026

The milestone was successfully reached, and the board was cleared in preparation for the next sprint. All issues were reviewed and moved to the “Done” column.

Milestone Review: Project Management As shown in burndown and burnup charts A.1 of the Project Planning A.3.1 milestone, all deliverables were fulfilled. However, the deadlines of the milestone had to be moved to the 10.03.2026, because the project planning required more time than initially expected. It should also be noted that the charts abruptly go down or up respectively, as we closed alot of the tickets after we reviewd them together just before we held this meeting.

Figure A.1.: Checklist and Burndown Sprint1

Sprint Retro 1: 10.03.2026

This sprint retro was held in Microsoft Whiteboard as it provides simple templates, the result is illustrated in A.2.

Key Takeaways:

- Maintain clear and consistent communication with the instructor.
- Be aware of maintaining a healthy work–life balance during the project.
- Apply lean project management practices to keep the process efficient and focused.

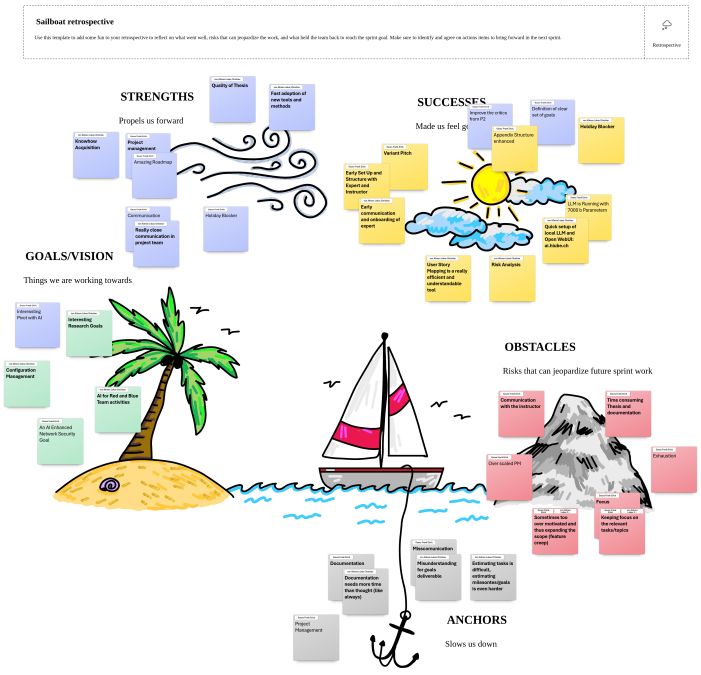

Figure A.2.: Sprint Retro 1

A.7.2. Sprint 2: 05.03 –18.03.2026

Sprint Planning 2: 05.03.2026

The current state of the project roadmap is shown in the following screenshot A.3.

Week	Responsible	Task
Week 1	Lukas	Configuration management implementation
Week 1	Frank	Research and comparison of frameworks
Week 1	Team	AI workshop on Thursday and writing of the related issues
Week 2	Team	AI research and evaluation
Week 2	Team	Preparation and pitch of the project deliverables

Table A.6.: Sprint Planning 2

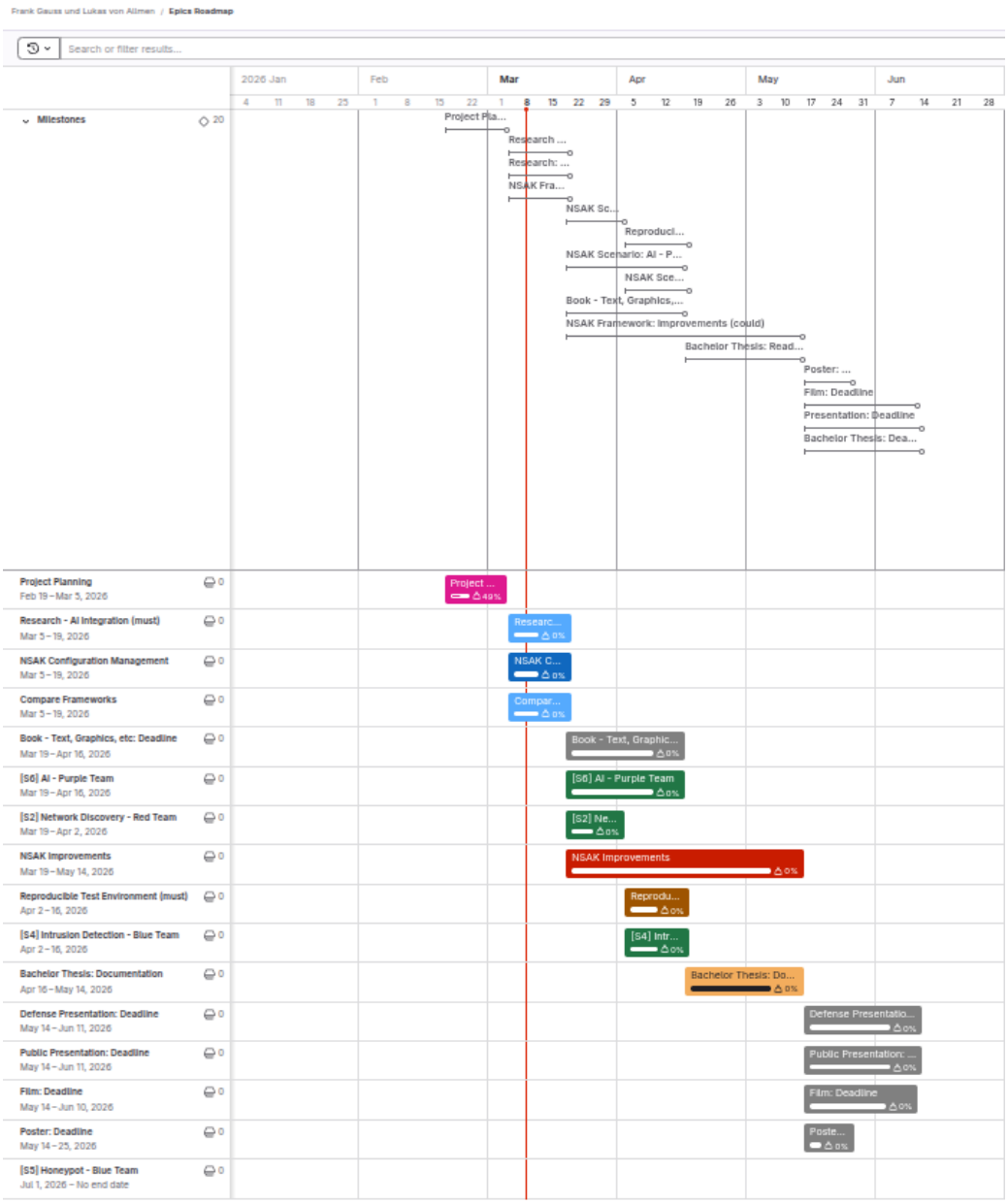

Figure A.3.: Roadmap Sprint Planning 2

Sprint Review 2: 19.03.2026

We got some helpful inputs related to both milestones from the tutor and expert in the checkpoint meeting A.8.3 earlier this day. We decided to still consider the milestone to be successfully reached as the planned work package and deliverables were finalized and presented. The resulting follow-up tasks are planned for

the next week and as a consequence we will hold the next sprint planning A.7.3 a week later than planned. All issues were reviewed and moved to the “Done” column in preparation for the next sprint. It must be noted that the burnup and burndown charts are not really representative, as the project team often reviews and closes the tickets at the end of the sprint.

Milestone Review: Compare Frameworks

The Compare Frameworks A.3.1 milestone is completed as illustrated in A.4.

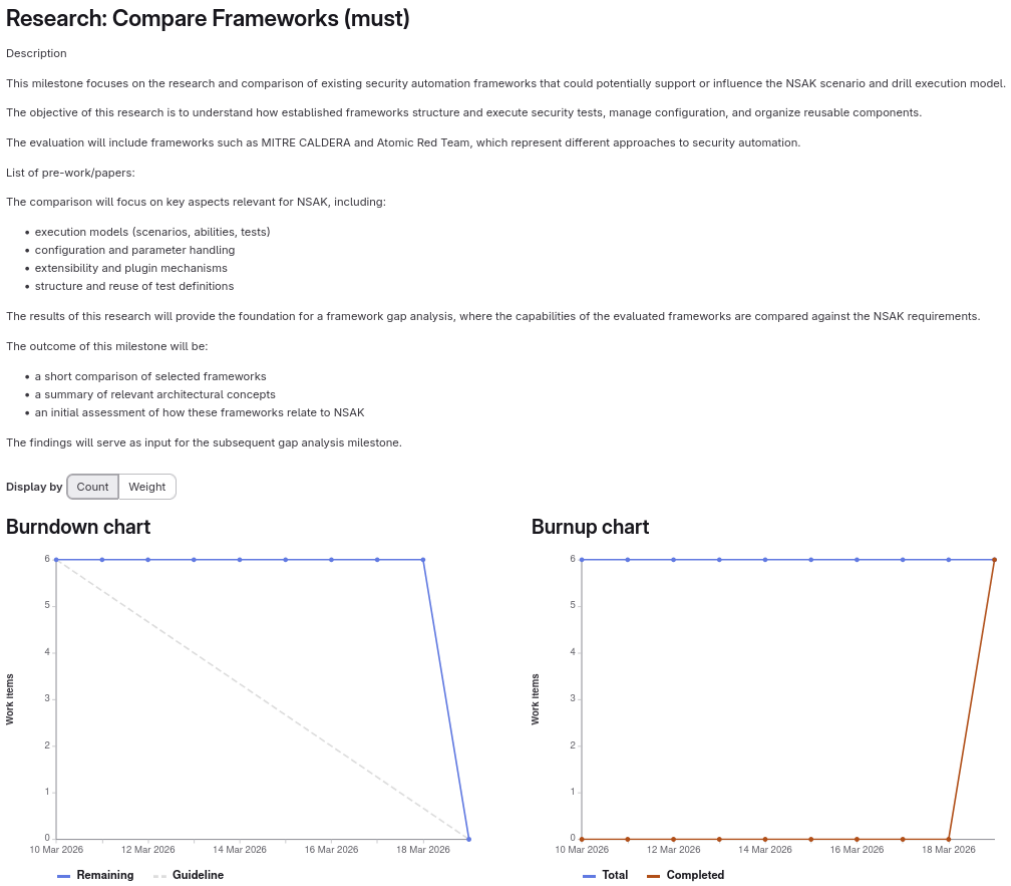

Figure A.4.: Milestone Description and Charts: Compare Frameworks

Milestone Review: Configuration Management

The burnup and burndown charts below A.5 are showing that the milestone Configuration Management A.3.1 is implemented as planned.

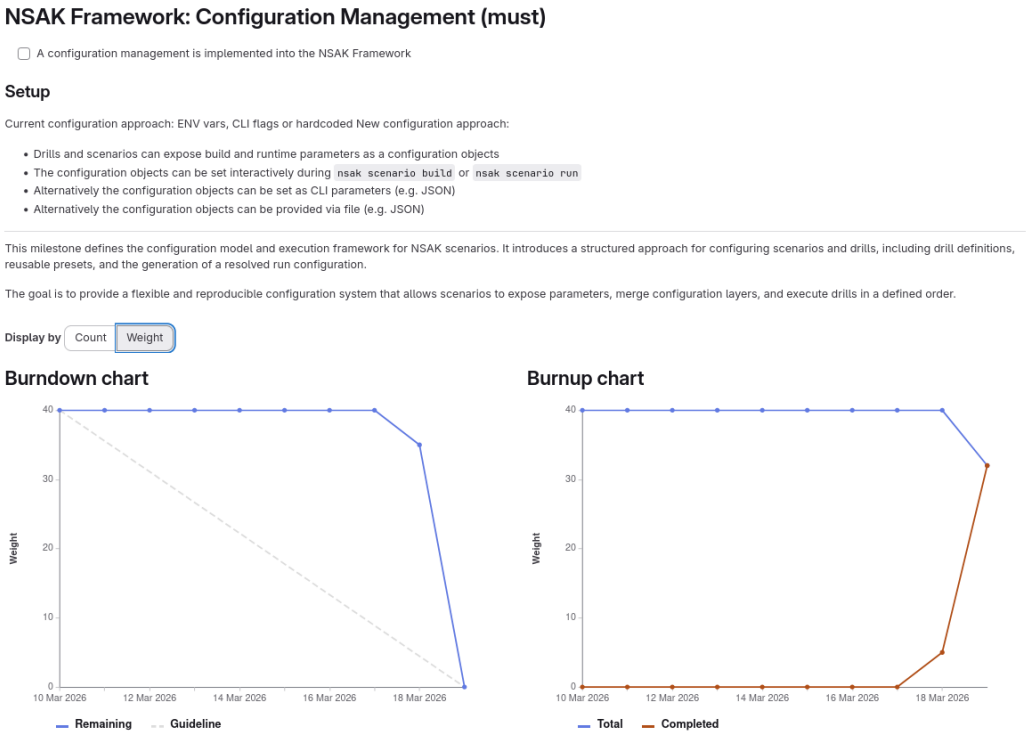

Figure A.5.: Milestone Description and Charts: Configuration Management

Sprint Retro 2: 19.03.2026

This sprint retro was held in Microsoft Whiteboard as it provides simple templates, the result is illustrated in A.6.

Key Takeaways:

- The long project planning phase greatly helped us with the projects overview and minimizes the planning needed for each sprint.
- The “focus weeks” really boosted the projects progress, and we are looking forward to the next “focus weeks”.
- Because we did not review tasks (Quality Assurance (QA)) on a regular basis, all this work needed to be done at the end of an iteration.
- Reviewing on a more regular basis would also lead to more accurate burndown and burnup charts.

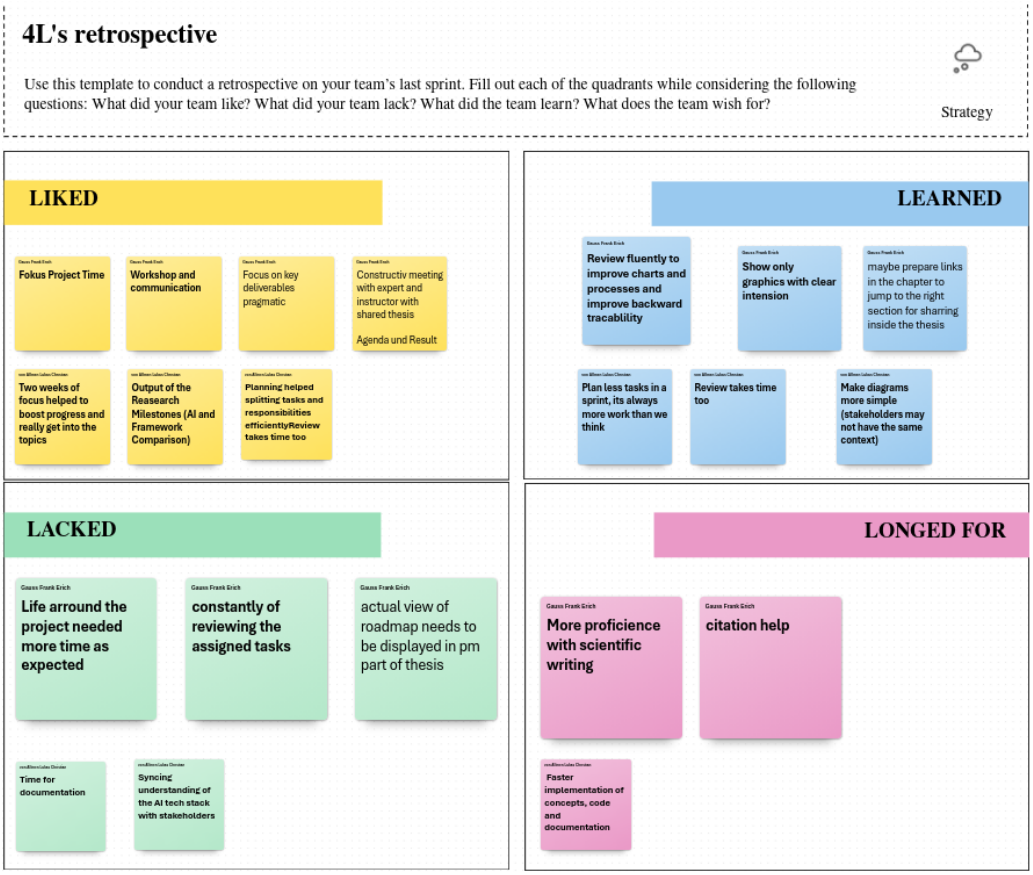

Figure A.6.: Sprint Retro 2

A.7.3. Sprint 3: 19.03 –01.04.2026

Sprint Planning 3: 26.03.2026

Because we got some important input regarding variant decision for AI-integration, framework comparison, and configuration management A.16 which led to additional work. As consequence of this, we decided to shorten the third sprint to one week, which is reflected in the following table A.7 and postponed the sprint planning from the 19.03 to the 26.03. To account for the delayed start in the overall planning we adjusted t as shown in the A.7.

Week	Responsible	Task
Week 1	Frank	Reproducible Test Environment implementation
Week 1	Lukas	NSAK Scenario: AI - Purple Team (first part)

Table A.7.: Sprint Planning 3

Sprint Review 3: 02.04.2026

Sprint Retro 3: 02.04.2026

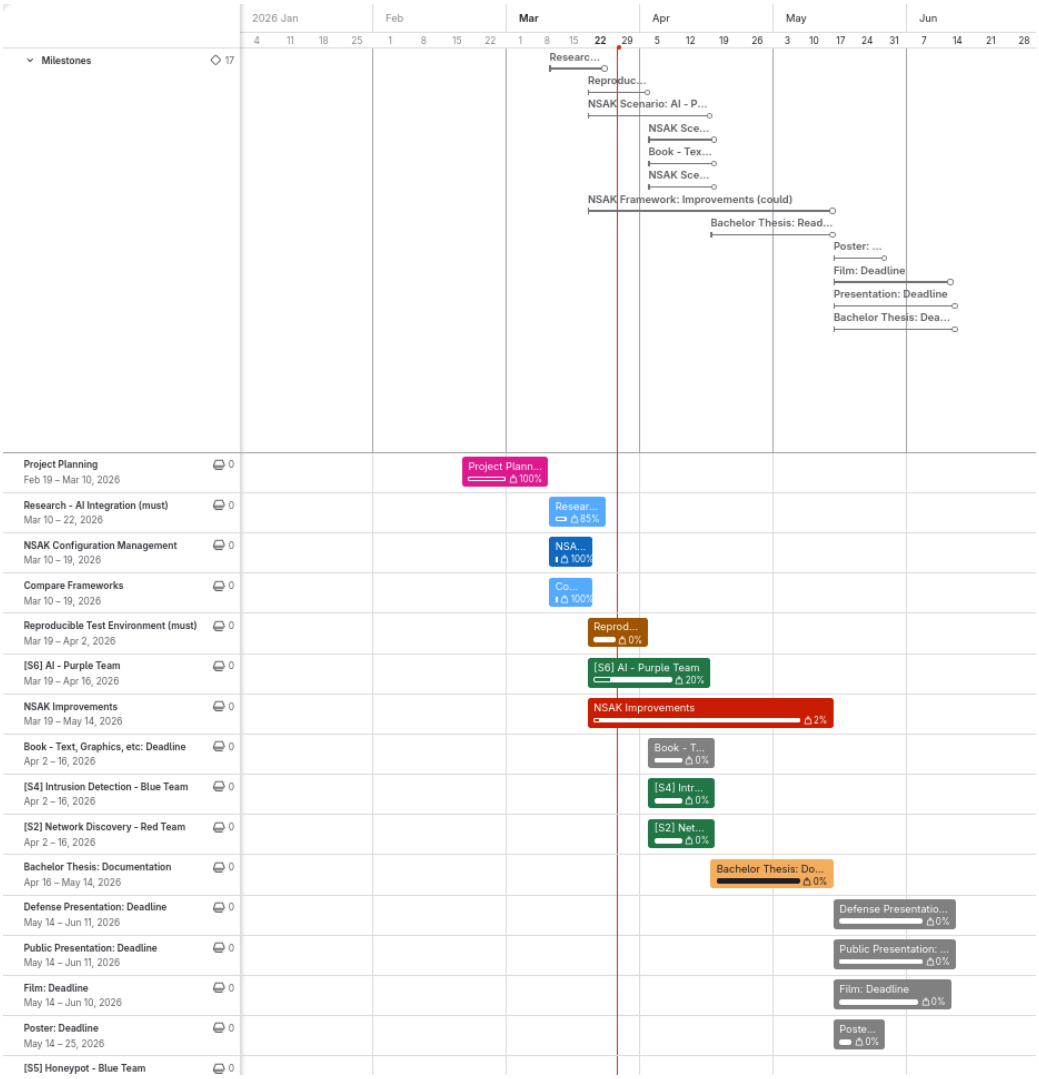

Figure A.7.: Roadmap Sprint Planning 3

A.7.4. Sprint 4: 02.04 –15.04.2026

Sprint Planning 4: 02.04.2026

Sprint Review 4: 16.04.2026

Sprint Retro 4: 16.04.2026

A.7.5. Sprint 5: 16.04 –29.04.2026

Sprint Planning 5: 16.04.2026

Sprint Review 5: 30.04.2026

Sprint Retro 5: 30.04.2026

A.7.6. Sprint 6: 30.04 –13.05.2026

Sprint Planning 6: 30.04.2026

Sprint Review 6: 13.05.2026

Sprint Retro 6: 13.05.2026

A.7.7. Sprint 7: 14.05 –27.05.2026

Sprint Planning 7: 14.05.2026

Sprint Review 7: 28.05.2026

Sprint Retro 7: 28.05.2026

A.7.8. Sprint 8: 28.05 –10.06.2026

Sprint Planning 8: 28.05.2026

Sprint Review 8: 11.06.2026

Sprint Retro 8: 11.06.2026

Rewrite in past tense

A.7.9. Risk Assessment

To methodically conduct the risk assessment the section “Risk assessment” from an online guide published by Advisera [20], which describes the risk assessment process according to ISO 27001, was used as a reference.

Risk Assessment Process:

1. Risk categories and criteria: Define which dimensions and taxonomies are important for the assessment.
2. Risk identification: Collect possible risks in a brainstorming session.
3. Risk analysis: Assess the risks criteria on a risk matrix.
4. Risk evaluation: Create a list of risks ordered by the resulting risk level and evaluate if they are acceptable or need treatment.
5. Risk treatment: Describe the strategy and how to treat the risks.

Risk Categories & Criteria:

The table A.8 shows four commonly used categories, which where used to group the risks.

Category	Description
Strategic Risk	Risks related to business decisions (e.g., wrong prioritization).
Operational Risk	Risks related to internal processes (e.g., Scrum, QA process).
Financial Risk	Risks related to costs and expenses (e.g., hardware costs).
External Risk	Risks related to uncontrollable sources (e.g., force majeure).

Table A.8.: Risk categories used for the project risk assessment

The criteria for analysing the risks are summarized in the table A.9

Criterion	Description
Probability	Likelihood that the risk will occur.
Impact	Severity of consequences if the risk materializes.
Level	Risk level calculated as Probability + Impact.

Table A.9.: Risk evaluation criteria

The levels of the criteria and their respective numerical weights are listed in the tables A.10 and A.11.

Level	Weight	Description
Low	1	Unlikely
Medium	2	Possible
High	3	Likely

Table A.10.: Risk probability scale

Level	Weight	Description
Low	1	Minor inconveniences
Medium	2	Delays milestones or reduces scope
High	3	Threat to the thesis

Table A.11.: Risk impact scale

The following table A.12 defines the calculation method and the resulting risk levels, used for the assessment.

Risk Level	Weight	Calculation
Very Low	2	Low + Low
Low	3	Low + Medium
Medium	4	Medium + Medium / Low + High
High	5	Medium + High
Critical	6	High + High

Table A.12.: Risk level classification (probability and impact are interchangeable)

Risk Identification

The identification process was conducted with the help of a digital brainstorming, where the risks are already grouped by category. The identified risks are represented by the cards shown in A.8. The respective risk categories are visualized by the color coding of the cards and described in the legend.

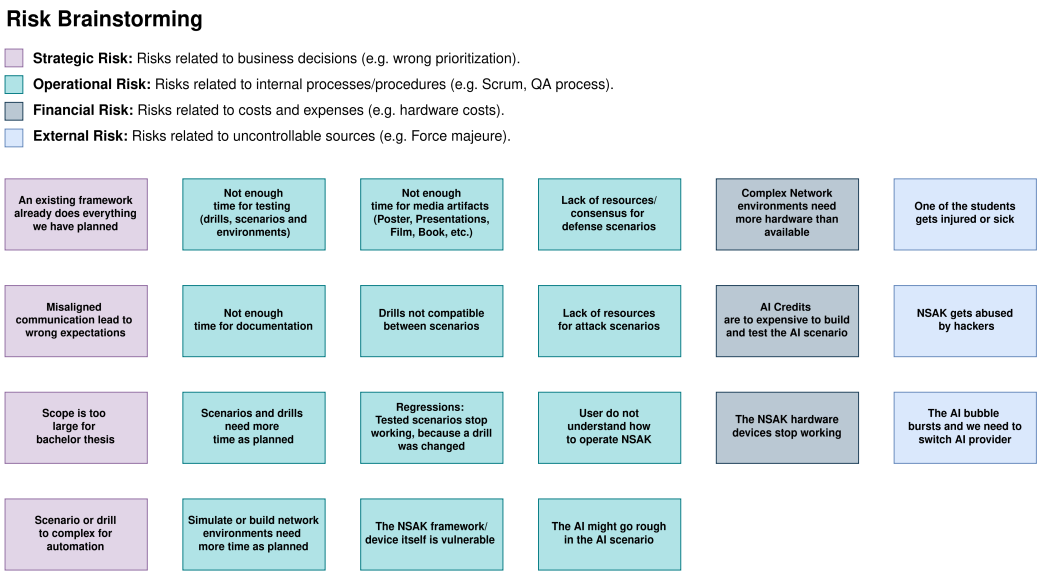

Figure A.8.: Risks Brainstorming

Risk Analysis

For analysing the risks a matrix was prepared, where the criteria “Impact” and “Probability” are representing the x- and y-axis respectively. The project team then assessed the criteria for each individual risk and placed them on the matrix accordingly. In a second step, the risks where moved if they did not fit relatively to each other. The resulting “Risk Analysis Matrix” is illustrated in the following figure A.9. The background color of the cells is representing the resulting risk level of the tasks.

Risk Matrix

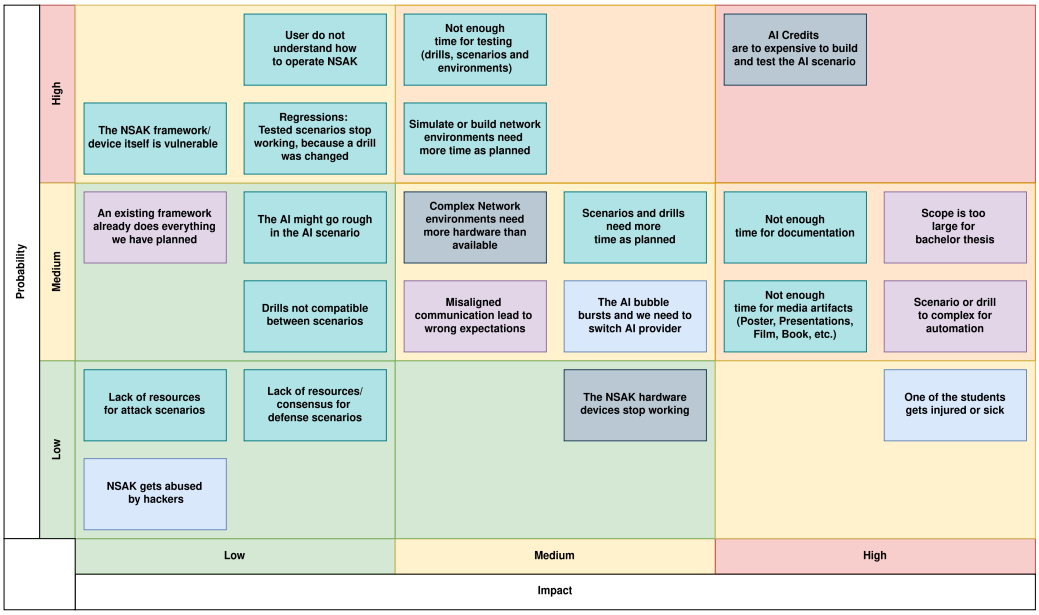

Figure A.9.: Risks Brainstorming

Risk Evaluation & Treatment

In a first step a table with the required columns was created. Then the risks and weights for the criteria where added with an ID in the format “R-000”, to easier reference them in the thesis later on. After sorting the table by the calculated risk level, the risk evaluation was conducted by deciding which treatment strategy described in A.13 we want to apply.

Strategy	Description
Mitigate	Reduce the likelihood or impact of a risk.
Transfer	Move the risk to a third party.
Accept	Acknowledge the risk and take no further action.
Avoid	Eliminate or circumvent the cause of the risk.

Table A.13.: Risk Treatment Strategies

Finally, the project team went through the risks and discussed how to treat them effectively. The resulting “Risk Assessment” is illustrated in the following table A.10.

Risk Assessment

Criteria: 1 = Very Low, 2 = Low, 3 = Medium, 4 = Medium, 6 = High, 9 = Critical

Level: 2 = Low, 3, 4 = Medium, 6 = High, 9 = Critical

Category: Strategic, Operational, Financial, External

Risk ID	Description	Risk Category	Probability	Impact	Level	Strategy	Treatment
R-001	AI credits are too expensive to build and test the AI scenario	Financial	3	3	9	Avoid	Use a self-hosted LLM, check if the BRH can provide AI services or if there are free plans for education
R-006	Scope is too large for bachelor thesis	Strategic	2	3	6	Mitigate	We have to plan and evaluate the goals carefully. The use of "User Story Maps" also helps to mitigate this risk.
R-007	Scenario or drill too complex for automation	Strategic	2	3	6	Mitigate	Create "User Story Maps" for epics, so that we have a good understanding of the scenarios and their drills.
R-002	Not enough time for documentation	Operational	2	3	6	Mitigate	Add a dedicated documentation issue with an according weight to each epic.
R-003	Not enough time for testing (drills, scenarios and environments)	Operational	3	2	6	Mitigate	Add a dedicated testing issue with an according weight to each epic.
R-004	Simulating or building network environments needs more time than planned	Operational	3	2	6	Mitigate	Add dedicated issues with according weight to each scenario epic.
R-005	User does not understand how to operate NSAK	Operational	3	2	6	Mitigate	Provide a Web GUI and/or user guides
R-012	Misaligned communication leads to wrong expectations	Strategic	2	2	4	Mitigate	Multiple channels and Bi-Weekly Meetings should keep the communication aligned.
R-010	Scenarios and drills need more time than planned	Operational	2	2	4	Mitigate	Use "User Story Maps" to get a good idea of the required steps to build the scenario, it's drills and estimate generously.
R-011	Not enough time for media artifacts (poster, presentations, film, book, etc.)	Operational	2	2	4	Mitigate	Include the creation of the media artifacts as milestones/epics in the project planning.
R-009	Complex network environments need more hardware than available	Financial	2	2	4	Avoid	We can simulate network environments instead of real hardware.
R-008	The AI bubble bursts and we need to switch AI provider	External	2	2	4	Avoid	Use vendor agnostic tools and protocols, so that we can switch provider easily.
R-014	Regressions: tested scenarios stop working because a drill was changed	Operational	3	1	3	Mitigate	Add automated test cases for detecting regressions early.
R-015	The NSAK framework/device itself is vulnerable	Operational	3	1	3	Transfer	Clearly state that the NSAK framework/device is still a POC and not ready for production use.
R-013	One of the students gets injured or sick	External	1	3	3	Accept	We accept this risk.
R-019	An existing framework already does everything we have planned	Strategic	2	1	2	Accept	We accept the risk, but will compare well known frameworks with NSAK and highlight our USP (HW and AI).
R-017	Drills not compatible between scenarios	Operational	2	1	2	Mitigate	The implementation of the configuration management should provide a way that small deviations can be branched.
R-018	The AI might go rogue in the AI scenario	Operational	2	1	2	Mitigate	Besides the fact that we have physical access to the device we add a killswitch to the AI scenario.
R-016	The NSAK hardware devices stop working	Financial	1	2	2	Accept	The fact that we have two devices already reduces the impact, further we could also use other HW.
R-021	Lack of resources for attack scenarios	Operational	1	1	1	Mitigate	Get familiar and use the MITRE ATT&CK framework as reference and source.
R-022	Lack of resources / consensus for defense scenarios	Operational	1	1	1	Mitigate	Get familiar and use well known sources such as NIST as source.
R-020	NSAK gets abused by hackers	External	1	1	1	Transfer	Clearly state that the NSAK framework/device is built only for educational and white hate purposes.

Figure A.10.: Risk Assessment

A.8. Meeting Notes

A.8.1. Checkpoint Meeting 1: 26.02.26

Add slides if we can reproduce them

Participants	Hj. Wenger (Instructor), Frank Gauss, Lukas von Allmen
Agenda	- Goals presentation via slides - Clarification whether there is a deadline or submission required for the presentation (besides adding it to the addendum). - Clarification whether the defense presentation should be an enhanced version of the public presentation or a separate, more technical presentation. - Proposal to establish a recurring meeting series every Thursday at 15:00 during the semester.
Tasks	- Hand in a set of success criteria for the project goals: We thought we can delivery the success criteria and the project goals as milestones in gitlab, we corrected this later on. - Conduct a risk assessment: A.7.9 - Contact expert via e-mail: Contacted at 21.02.26, response 03.03.26, invited to next checkpoint meeting 05.03.26, the expert takes part at the meeting every two weeks

Table A.14.: Meeting Notes: Checkpoint 1

A.8.2. Checkpoint Meeting 2: 05.03.26

Add slides if we can reproduce them

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Review of the project goals as milestones and project planning. - Review of risk assessment. - Discussion on the investigation and analysis of network communication between end devices and external networks (RED scenario: sniffing, filtering, and potential data exfiltration).
Decisions	- Variant 1 was selected for further development. - Open questions remain regarding the use and management of AI tokens.
Notes	- We misunderstood how to deliver the project goals and success criteria: Instead of defining the goals as milestone we added a goal section in the thesis. - Discussion on the rationale behind selecting Variant 1. Consideration of whether a user interface is required when MCP functionality is available - Open Research Question. - NSAK integrates AI capabilities. - The development of a Web GUI is currently considered lower priority - suggestion of Urs Keller.
Tasks	- Send the slides of the final presentation and documentation of “Project 2” to Urs Keller (Expert): “Done” (via E-Mail). - Delivery the projects goals with a detailed description as part of the thesis by Saturday, 07.03.26: “Done” 1.3 - Finalize the project planning: “Done” A.6 - Write the first few chapters of the thesis: “Done”

Table A.15.: Meeting Notes: Checkpoint 2

A.8.3. Checkpoint Meeting 3: 19.03.26

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Review Project Management and Roadmap - Review of the active milestones (Framework Comparison, AI-Research, Configuration Management) - Discuss Variant Decisions: AI-Integration - Ask about AI token usage - Ask about repository access
Decisions	- We will proceed using NSAK as the basis for our thesis. - Open questions remain regarding the use and management of AI tokens.
Notes	- -
Tasks	- Document decision that we will continue to work with the NSAK framework: 2.2.3 - Expand on variant decision for AI-Integration: - Split diagram into 2 variants: - Expand in detail how the variants are working (e.g., sequence diagram): A.14 and A.16 - Configuration management: Move metadata and others below the device node: “Done” 5.1.2 - BFH TI: Has a cluster with studio macs for AI usage -> Ask if we can use them (https://mlmp.ti.bfh.ch/ via VPN)

Table A.16.: Meeting Notes: Checkpoint 3

A.8.4. Checkpoint Meeting 4: 02.04.26

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Overview Roadmap and active milestones - Reproducible Test Environment: Introduce ContainerLab, Show diagram, Physical Implementation (could) - AI Scenario: Demo AI-agent connection to self-hosted, LangChain Research LLM - Review MCP Sequence Diagrams - Discuss what needs to be added to the main document and what to the appendix - Ask if it's possible to lend more HW for physical test setup - Ask for access to BFH AI Cluster
Decisions	- -
Notes	- - Alternative to container LAB: Cisco - DNS3 (not open source) - It would be interesting to add more services to the DMZ in the reproducible test environment - Tutor: The current structure is preferred, we should also ask the expert -
Tasks	- Ask for access to BFH AI: https://mlmp.ti.bfh.ch (Requires VPN) - If we are not allowed, ask tutor for guidance - Variant decisions: Add pros/cons and elaborate why we choose a variant - AI-agent: Design and implement logging - Ask the expert about the preferred document structure: Thesis vs Appendix - Create a HW list for tutor, date and configuration (must be planned, device configuration could be done by assistants) -

Table A.17.: Meeting Notes: Checkpoint 4

A.8.5. Checkpoint Meeting 5: 16.04.26

A.8.6. Checkpoint Meeting 6: 30.04.26

A.8.7. Checkpoint Meeting 7: 14.05.26

A.8.8. Checkpoint Meeting 8: 28.05.26

A.8.9. Checkpoint Meeting 9: 11.06.26

Variant 1: AI Purple Team

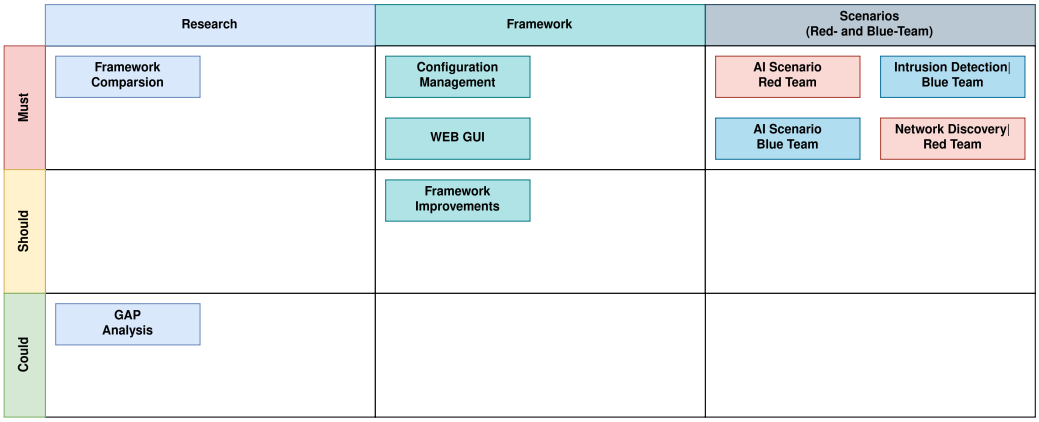

Figure A.11.: Variant 1: AI vs Manually Built Scenarios

A.9. Variant Decision: Thesis Goals

During the first sprint of the project, which was focused on project management and planning, we identified the very promising hypothesis that red and blue team activities could be automated and/or enhanced by AI. Pursuing this hypothesis would change the projects focus and goals and thus requires the agreement and approval of the stakeholder. To communicate this circumstance efficiently and leave the option to go with the status quo, we prepared two options to conduct a "Variant Decision". In the following two subsections the two variants presented in the second checkpoint meeting A.15 are described and illustrated. It must be noted, that the variants and their visualizations A.11 A.12 are momentary snapshots and will not be updated for traceability reasons. For example, the Web Graphical User Interface (GUI) was already deprioritized during the discussion of the variants, which is reflected in the project goals 1.3 but not here.

A.9.1. Variant 1: AI vs Manually Built Scenarios

Figure A.11 shows the project goals adjusted for the AI focused hypothesis, which is favored variant of the project team. In this option most of the evaluated scenarios will not be realized in favor of AI based scenarios, which also requires additional research 1.3.1. The basic idea is to implement scenarios for network discovery 1.3.3, intrusion detection 1.3.3 and an AI based scenario with the same capabilities 1.3.3. In the evaluation of the thesis we can then assess the quality of the AI based scenarios itself 1.3.4 and in comparison to the manually built scenarios 1.3.4.

Variant 2

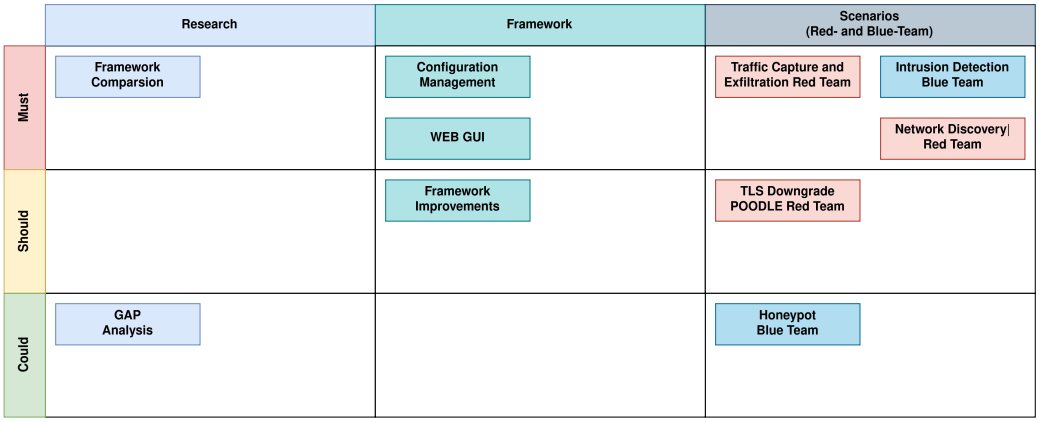

Figure A.12.: Variant 2: Multiple Blue and Red Team Scenarios

A.9.2. Variant 2: Multiple Blue and Red Team Scenarios (Status Quo)

This is the fallback if the other option A.9.1 is not approved by the stakeholders. Even if the project team does not favor this more conservative variant, it still represents a valid way to conduct an interesting project. As illustrated below A.12, in this variant we propose to proceed with the previously planned scenarios for which we already conducted "User Story Mappings" documented under the workshops sectionA.11. The focus would lie in more conventional scenarios, overall framework improvements and an in depth comparison with another framework in form of a GAP Analysis.

A.9.3. Decision

The stakeholders and project team decided in favor of **Variant 1: AI vs Manually Built Scenarios** A.9.1 in the checkpoint meeting A.15.

A.10. Variant Decision: AI Integration

During the AI-research 2.2 we identified two interesting ways to allow NSAK to interact with AI based technologies. As illustrated in A.13, the first variant purposes to implement an AI-agent directly into the framework and use it in scenarios and drills. In the second variant shown in A.15, the capabilities of NSAK and other relevant tools on the NSAK device would be exposed via MCP.

A.10.1. Variant 1: AI-agent in NSAK

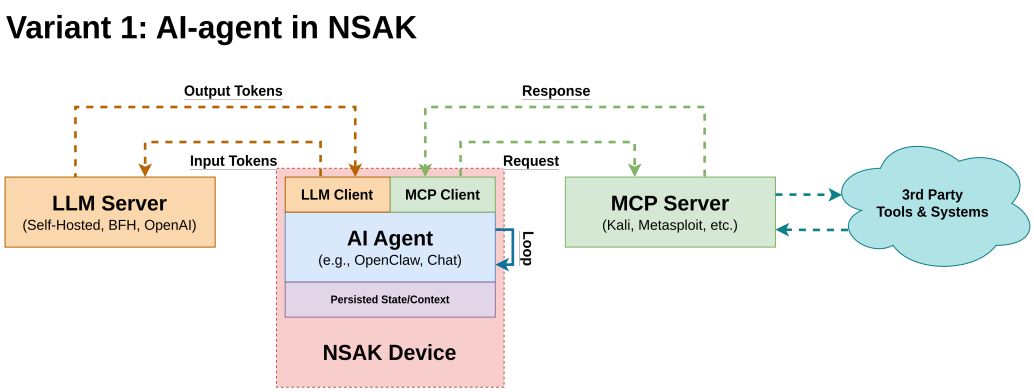

Figure A.13.: AI Integration Variant 1: AI-agent in NSAK

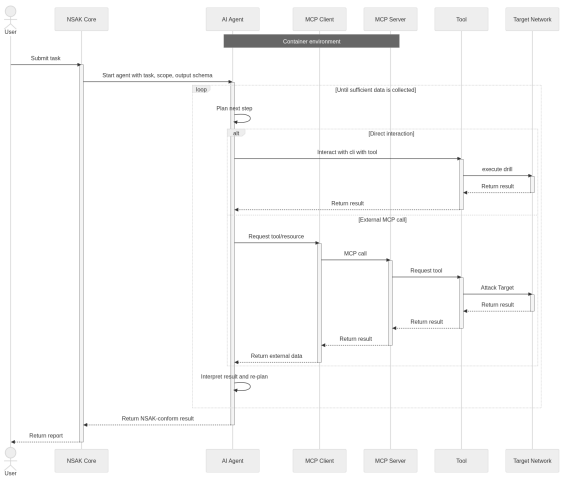

Figure A.14.: Variant 1, AI Agent in NSAK

In this architecture, the AI agent is embedded within the NSAK device. The NSAK core establishes the operational guardrails and schema, and provides the initial

prompt to the AI Agent. The agent uses a local or remote LLM, which is accessible over the management network. It operates within an iterative loop, in which it may either invoke external tools via a command-line interface (e.g., nmap, Metasploit) or interact with them through an MCP server interface. The results of these operations are subsequently processed and returned to the NSAK core in a structured schema-compliant format.

A.10.2. Variant 2: NSAK as MCP Server

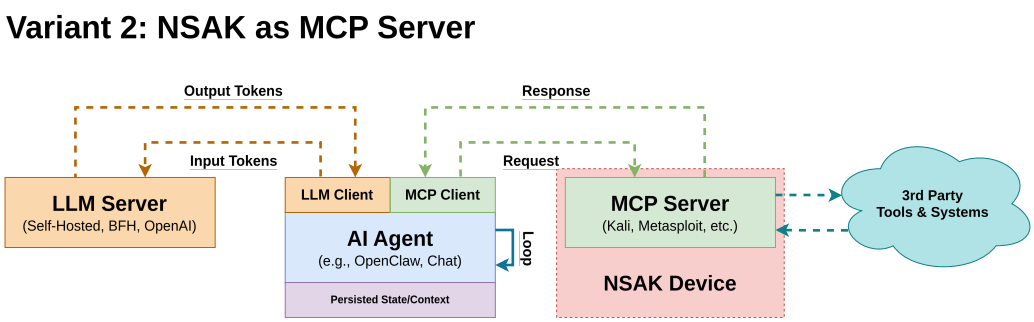

Figure A.15.: AI Integration Variant 2: NSAK as MCP server

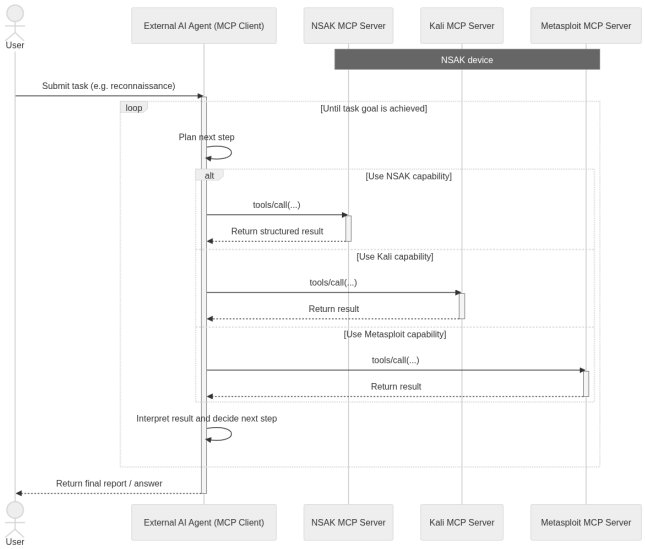

Figure A.16.: Variant 2 NSAK MCP Server

Variant 1, in which the Agentic-AI is embedded within the NSAK device, enables the definition and enforcement of schemas and security policies within a controlled environment. Furthermore, it allows direct interaction with tools via the

command-line interface and integration with MCP servers. This results in a tightly coupled architecture with high control and consistency.

In contrast, Variant 2 relies on an external agent that accesses the NSAK device through an exposed MCP server interface. While this approach provides increased modularity, it also introduces limitations in flexibility, as the agent is restricted to the set of tools exposed by the NSAK device.

From a holistic and multi-layered architectural perspective, Variant 1 is preferred. It enables deeper system integration, higher security, and a wider range of executable tools.

A.10.3. Decision

The project team decided in favor of **Variant 1: AI-agent in NSAK** A.10.1, as it promises to greatly increase the capabilities of NSAK regarding autonomy and automation.

A.11. Workshops

In the Project Management phase, several workshops were conducted, including a brainstorming session using Post-it notes and a User Story Mapping exercise on a whiteboard. The goal of these workshops was to clarify the scenarios a network edge device should support.

Following these workshops, the insights gathered were used to derive issues and goals, which were then mapped to detailed milestones in GitLab. All deliverables were categorized using the MoSCoW prioritization method into must, should, and could requirements. An example of this prioritization and scenario mapping is illustrated in Figure A.19. All Milestones links are provided in the URL at the top of the section.

A.11.1. Brainstorm Scenario Selection

In this workshop, participants brainstorm and cluster potential red and blue team scenarios for practical cybersecurity exercises.

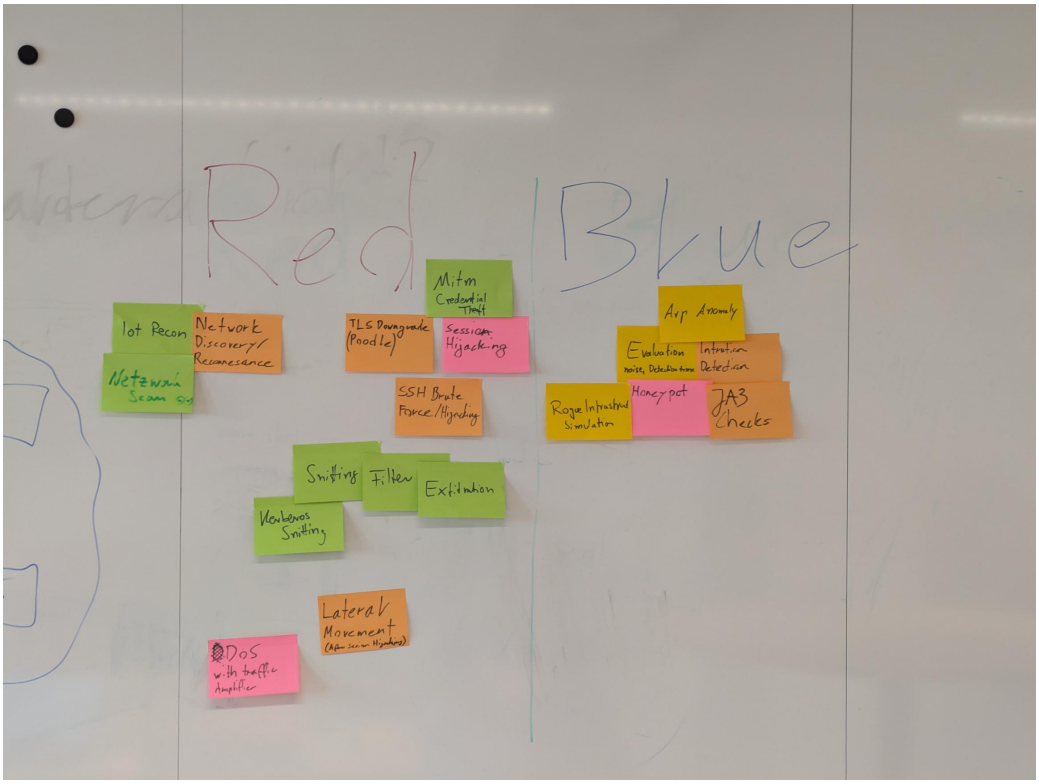

Figure A.17.: Brainstorm Scenarios

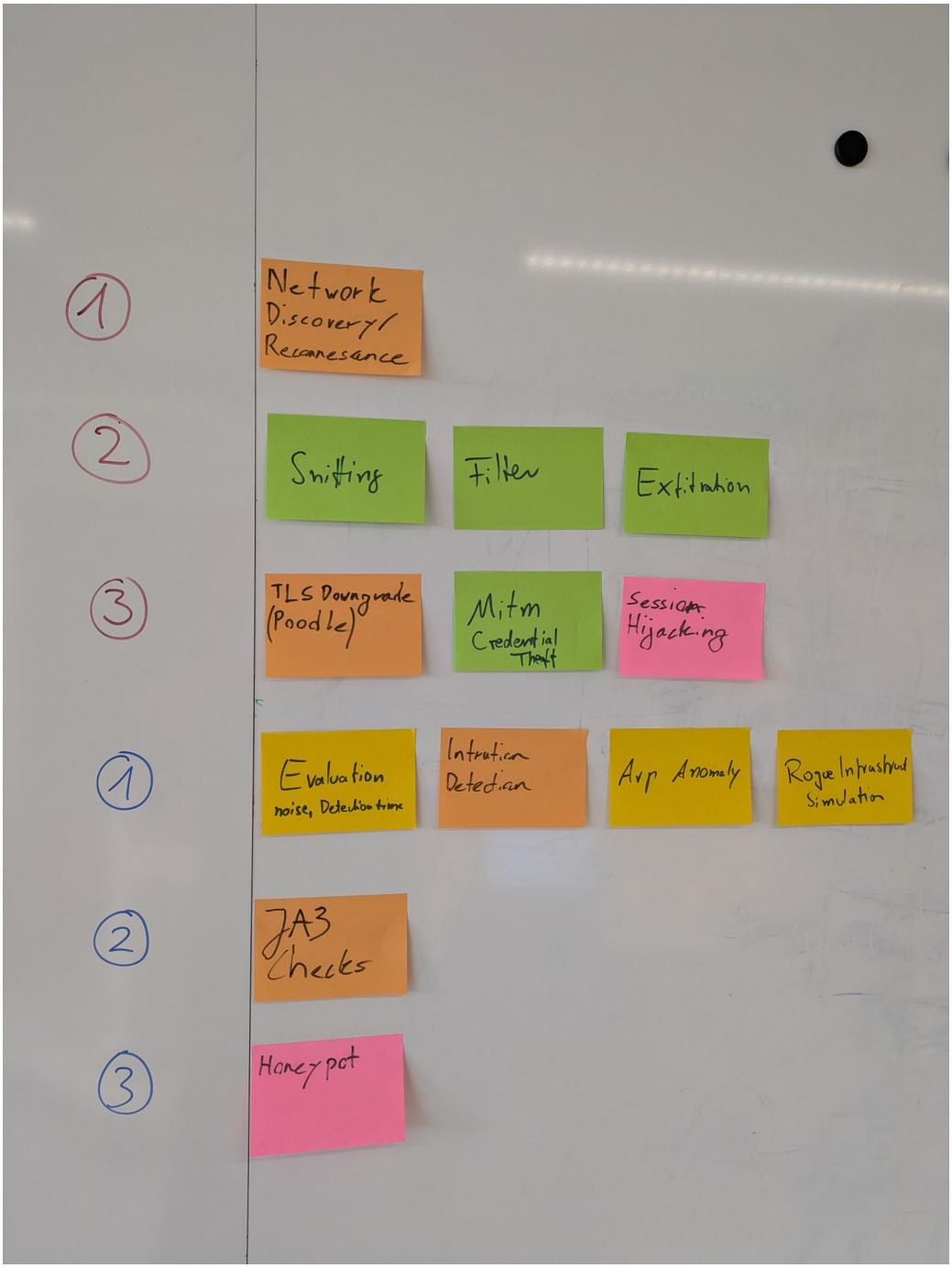

Figure A.18.: Clustering possible Scenarios

The Brainstorm session is used to derive further red/blue team scenarios and led to the pivot of AI-enabled Nsak scenario.

A.11.2. NSAK Scenario: Network Discovery - Red Team

Network Discovery Scenario

Objectives

This workshop aims to evaluate the different steps of the network discovery process and to help participants understand how attackers identify and map potential targets within a network.

Building on this objective, Red Teams consider network discovery one of the most essential activities during the reconnaissance phase, as it helps them identify hosts and services and evaluate potential targets. Once a network map is created, the team can use it in later phases. For example, they might execute an ARP spoofing attack, which allows them to intercept network traffic more efficiently.

In relation to these activities, this scenario addresses the following point from the thesis proposal:
'Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen.'

The workshop led to the milestone *NSAK Scenario: Network Discovery – Red Team (must)*, in which each step for discovering and mapping a network is documented and structured for later implementation.

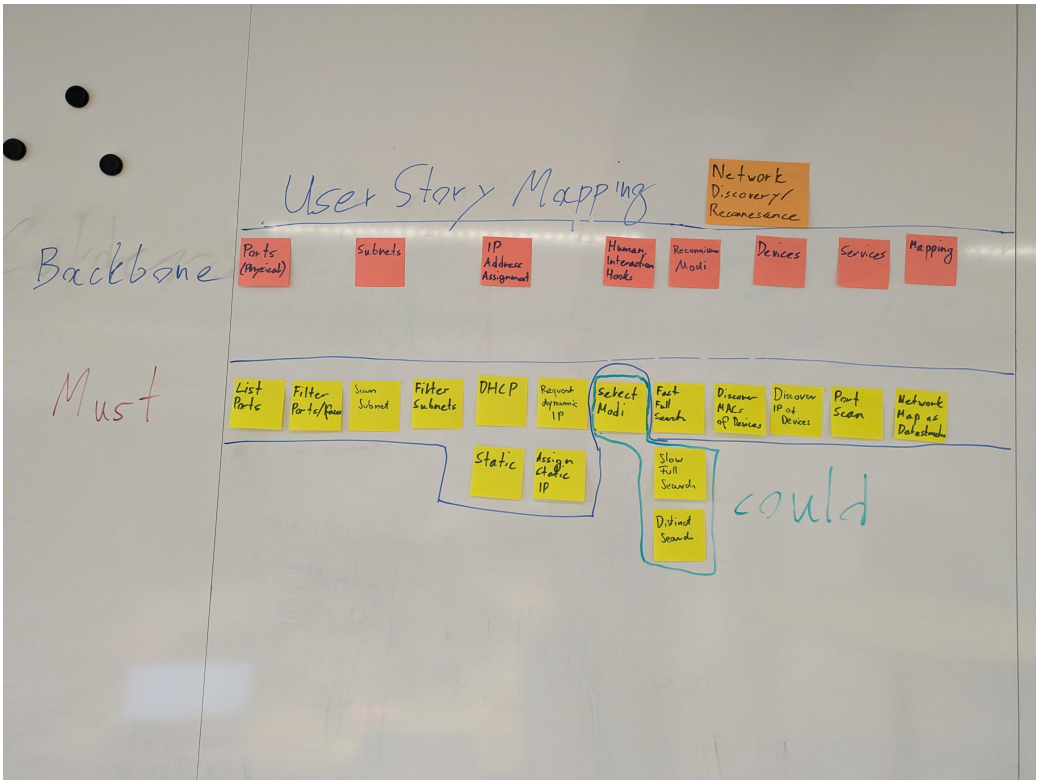

Figure A.19.: NSAK Scenario: Network Discovery - Red Team

A.11.3. NSAK Scenario: TLS Downgrade Poodle - Red Team

TLS Downgrade Poodle Milestone

Objectives

This workshop demonstrates how attackers can exploit TLS downgrade vulnerabilities, such as POODLE, to intercept encrypted communication.

In this scenario, the infamous POODLE attack is implemented, in which a malicious actor tries to convince a client and a server to use the vulnerable TLS version 1.1 instead of 1.2 or 1.3 to encrypt HTTP traffic. This attack requires the attacker to sit between the client and the server (a man-in-the-middle, or MITM), allowing them to intercept and control the traffic. When the client initiates a TLS handshake, the attacker drops packets to the server and sends TCP reset packets to the client, eventually trying older TLS versions. Because TLS 1.1 is vulnerable to a CBC Padding attack, it is possible to create a padding oracle that gradually decrypts the cookie and hijacks the user's session.

Building on this, the scenario directly addresses the following aspect from the thesis proposal:
‘Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen’

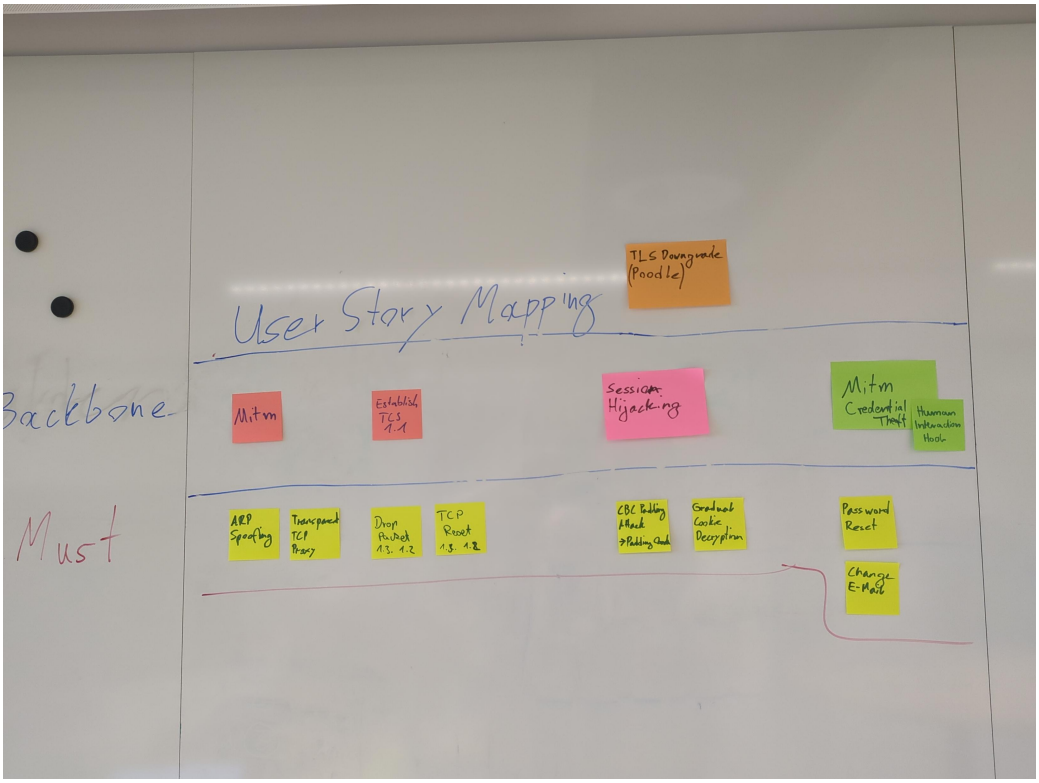

Figure A.20.: NSAK Scenario: TLS Downgrade Poodle - Red Team

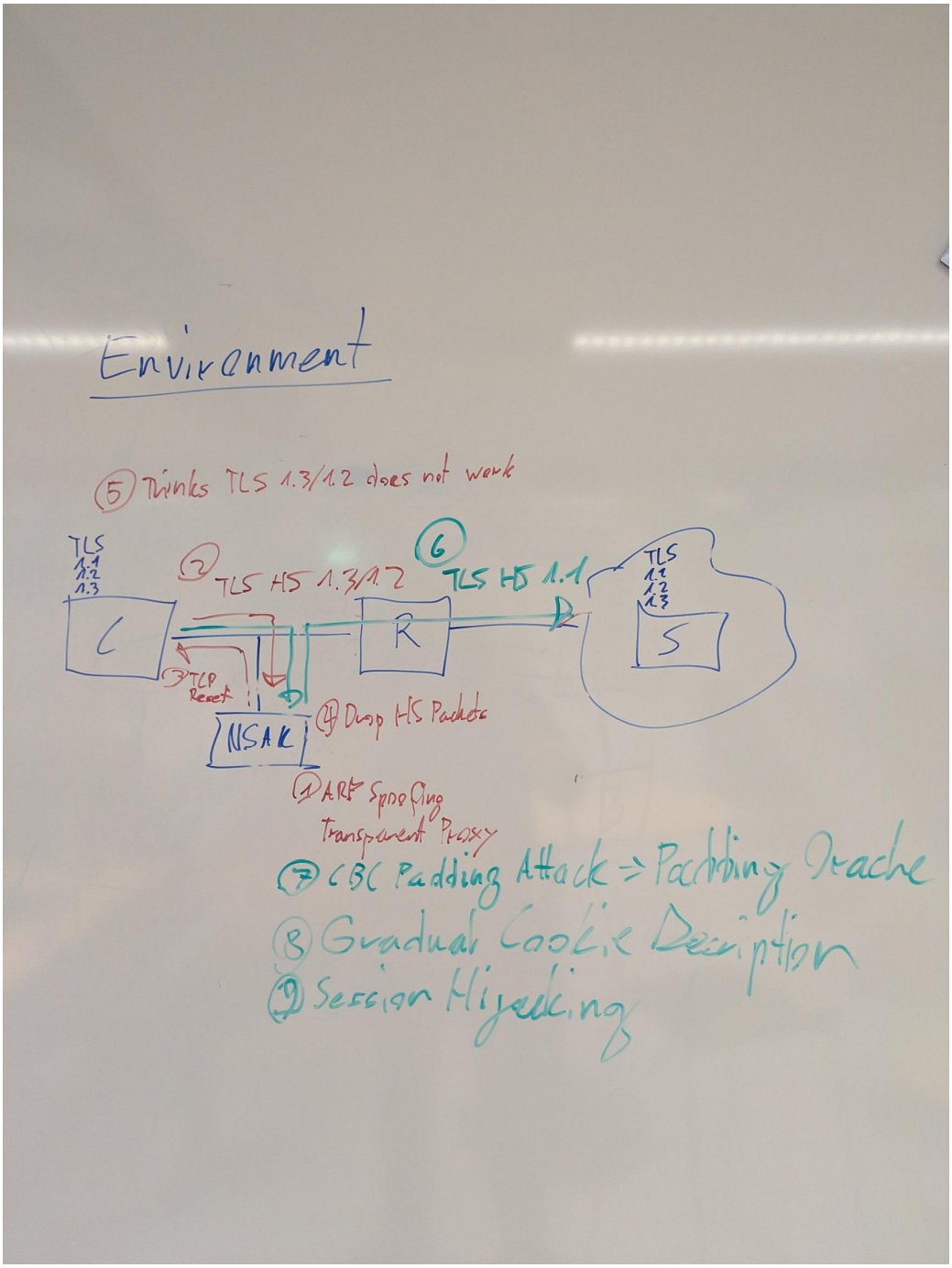

Figure A.21.: NSAK Scenario: TLS Downgrade Poodle -Topology

A.11.4. NSAK Scenario: Honeypot - Blue Team

Honeypot Milestone

Honeypots can serve as an additional form of intrusion detection and help Blue Teams detect attackers in a network. The system tries to attract malicious actors by providing an attack surface as large as possible; a simple way to achieve this is to open ports. After an attacker tries to connect to any of the open ports, the system notifies an operator and may close all open ports to prevent further attempts to compromise the Honeypot. The operator can then manually intervene to identify and isolate the attacker and analyze the captured payload.

This scenario covers the following bullet point from the thesis proposal:
‘Bereitstellung von Funktionen zur Erkennung, Protokollierung und Analyse verdächtiger Aktivitäten als Teil defensiver Massnahmen’

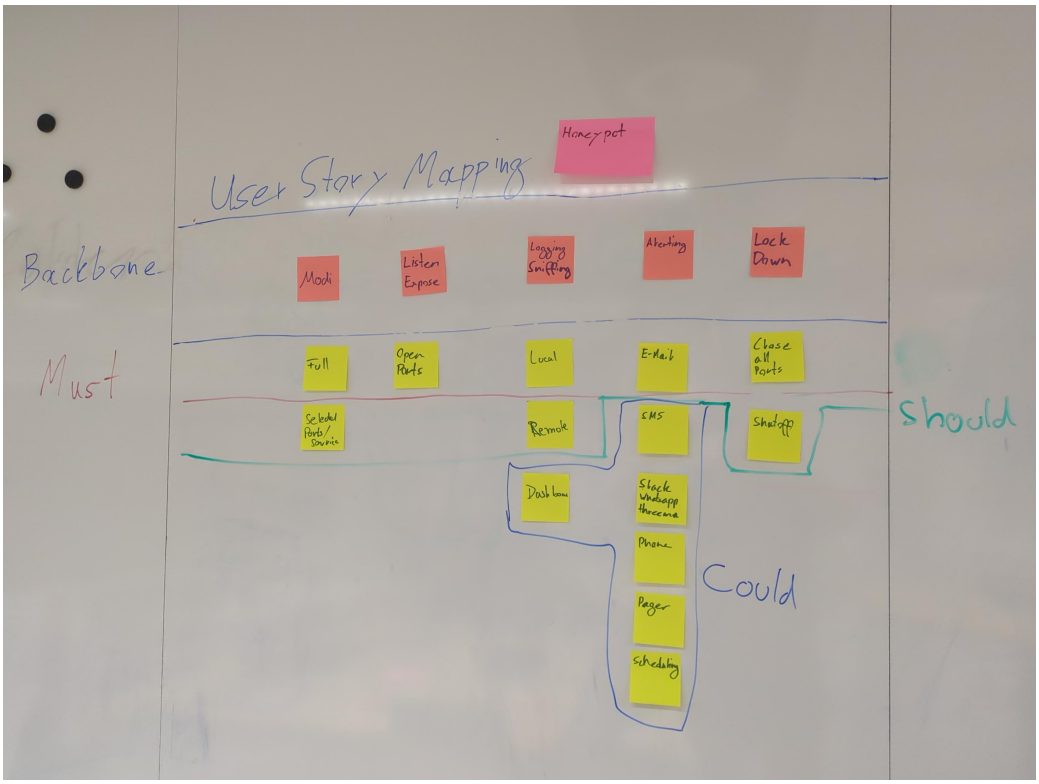

Figure A.22.: NSAK Scenario: Honeypot - Blue Team

A.11.5. NSAK Scenario: Traffic Capture - Red Team

Traffic Capture Milestone

Capturing, exfiltrating, and analyzing traffic from a targeted network is a key Red Team activity and crucial for initial reconnaissance and planning the next phases of the attack. Based on the gathered intelligence, network participants and services can be identified, and possible attack vectors may be evaluated. If unencrypted traffic is present on the network, session data and credentials could already have been extracted.

This scenario addresses the following bullet point from the thesis proposal: 'Untersuchung und Auswertung der Netzwerkkommunikation zwischen Endgeräten und externen Netzen.'

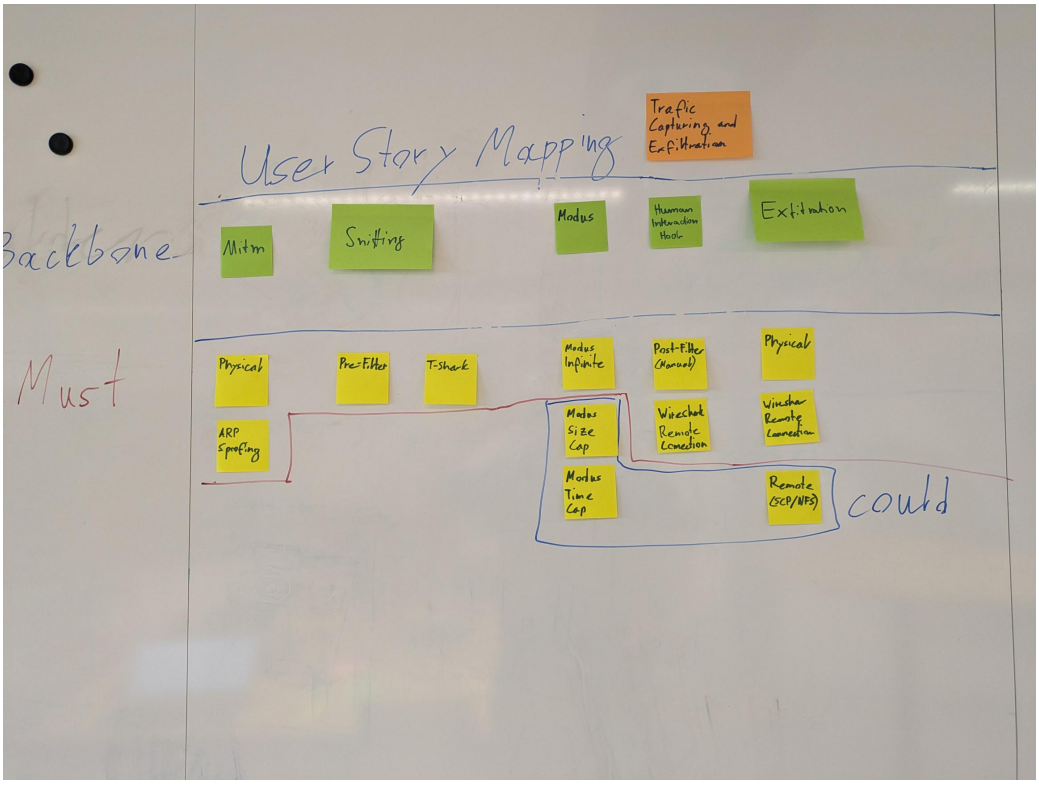

Figure A.23.: NSAK Scenario: Traffic Capture - Red Team

A.11.6. NSAK Scenario: Intrusion Detection - Blue Team

Intrusion Detection Milestone

Intrusion detection is one of the core Blue Team activities and aims to identify malicious actors and misconfigured clients in production network environments. This can be achieved by introducing a system at a traffic mirroring port or as a central point in a network. The intrusion detection system can leverage a wide array of security policies, reference traffic, and other markers to effectively iden-

tify anomalies. As soon as suspicious activity is detected or a policy is violated, automated measures may be initiated, and security engineers will be automatically alerted for manual intervention.

This scenario covers the following bullet point from the thesis proposal: “Bereitstellung von Funktionen zur Erkennung, Protokollierung und Analyse verdächtiger Aktivitäten als Teil defensiver Massnahmen”

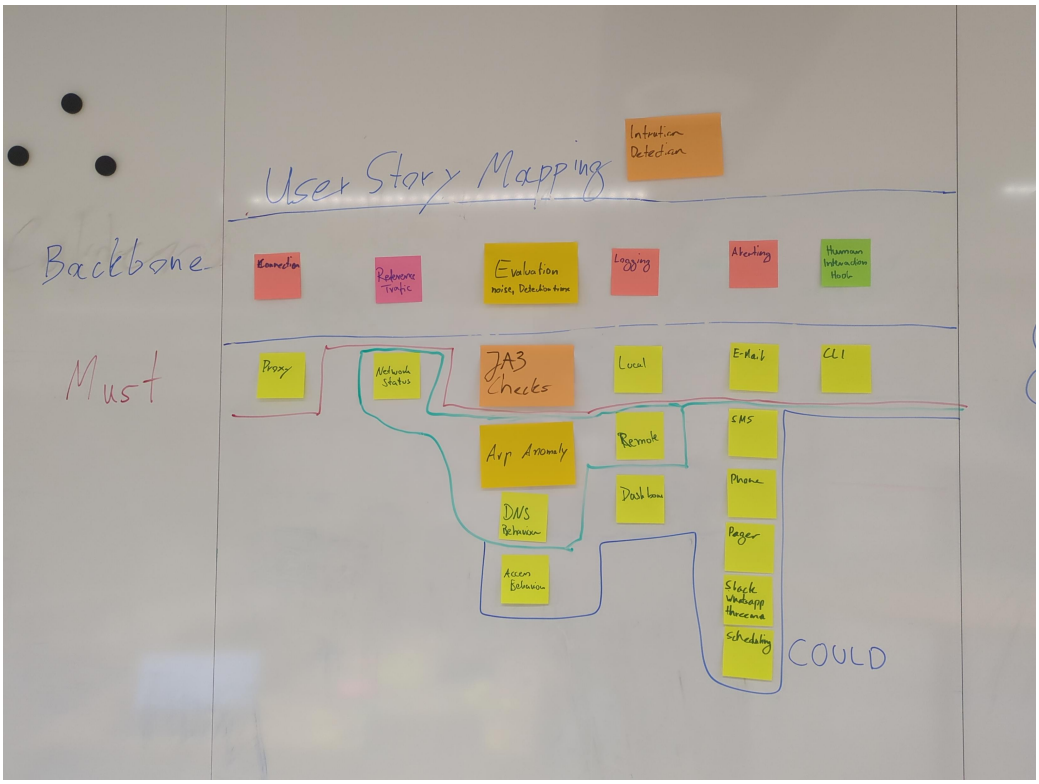

Figure A.24.: NSAK Scenario: Intrusion Detection - Blue Team

A.12. Self-Hosted LLM

During the risk assessment A.7.9 we identified that the AI tokens required to use cloud based LLMs might be very expensive, and it's really hard to estimate the costs. One of the possibilities to avoid this risk was the usage of self-hosted LLMs, and luckily we already had existing hardware at home which is suitable for running smaller LLMs. Besides eliminating the costs for AI tokens, at least during the development phase, it also allows the quick evaluation of different AI Models. Depending on the quality and performance of the smaller models running in the self-hosted setup we can consider not using a cloud based LLM at all.

A.12.1. Hardware Setup

Component	Specification
CPU	AMD Ryzen 5 3600 (6 cores / 12 threads, up to 4.2 GHz)
GPU	NVIDIA GeForce RTX 3070 (8 GB VRAM)
System Memory	32 GB RAM

Table A.18.: Hardware configuration of the self-hosted LLM inference server.

The 8 GB VRAM of the RTX 3070 allows us to run models with up to 7 billion parameters, depending on the specific model [21].

A.12.2. System Setup

The system setup described in 15 will configure and expose the Ollama API and OpenWeb UI via NGINX, which provided a chat GUI to interact with Ollama. In contrast to Ollama, which runs directly on the host, the other services are run in containers managed by rootles podman and defined the docker-compose.yml 16. The Nginx default.conf 17 configures the web server to terminate the SSL connection and act as reverse proxy for the "Open WebUI" and the Ollama API under the directory `/llm/`.

Services:

- Ollama: LLM manager, exposing an API for prompts and other interactions (host).
- Swag: Nginx web server with Certbot for automated SSL certificate generation (containerized).
- Open WebUI: Web based chat interface for LLM managers (containerized).

```
# Login as default user in sudoers or root user

# Check nvidia driver is loaded and the GPU was correctly detected, otherwise install the according drivers
nvidia-smi

# Install Ollama, podman, podman-compose and enable systemd service
sudo pacman -Syu ollama ollama-cuda podman podman-compose
sudo systemctl enable --now ollama
sudo systemctl status ollama

# Download and run a qwen coder 7b model for testing
ollama pull qwen2.5-coder:7b
ollama run qwen2.5-coder

# Create a new user for isolating the services
sudo useradd -m ai
sudo passwd ai

# Enable linger for user ai, required to run services such as rootles containers without being logged in
sudo loginctl enable-linger ai

# Edit the sysctl.conf file
sudo vim /etc/sysctl.conf

# Add the following lines, required for rootless podman to use the ports 443 and 80
net.ipv4.ip_unprivileged_port_start=443
net.ipv4.ip_unprivileged_port_start=80

# Load the new config
sudo sysctl -p

# Login as user ai and create the following directory and file
mkdir -p .config.containers
vim .config/containers/containers.conf

# Add the following lines, required for rootless podman (podman mangages cgroups directly instead of systemd)
[engine]
cgroup_manager="cgroupfs"

# Create a docker compose file and add contents from lst:nginx-openwebui-docker-compose-yml
vim docker-compose.yml

# Start the services, so that they create the volume mappings etc, the containers will probably fail for now
podman-compose up -d
podman-compose down

# Open the nginx default.conf and add the contents from
```

```yaml
services:
  swag:
    image: lscr.io/linuxserver/swag:latest
    container_name: swag
    cap_add:
      - NET_ADMIN
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/Zurich
      - URL=hiube.ch
      - VALIDATION=http
      - SUBDOMAINS=ai,
      - EMAIL=<my-email>
      - ONLY_SUBDOMAINS=true
      - EXTRA_DOMAINS= #optional
      - STAGING=false #optional
      - DISABLE_F2B= #optional
      - SWAG_AUTORELOAD= #optional
      - SWAG_AUTORELOAD_WATCHLIST= #optional
    volumes:
      - /home/ai/swag:/config
    ports:
      - 443:443
      - 80:80 #optional
      - 443:443/udp #optional
    restart: unless-stopped

  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    volumes:
      - /home/ai/open_webui:/app/backend/data # Use a named
        ↪  volume for persistence
    environment:
      OLLAMA_BASE_URL: http://host.docker.internal:11434
    restart: always
```

Listing 16: /home/ai/docker-compose.yml

```
# redirect all traffic to https
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name ai.hiube.ch;

    location / {
        return 301 https://$host$request_uri;
    }
}

# main server block
server {
    listen 443 ssl default_server;
    listen [::]:443 ssl default_server;
    server_name ai.hiube.ch;
    include /config/nginx/ssl.conf;
    root /config/www;
    index index.html index.htm index.php;

    # enable subfolder method reverse proxy confs
    # include /config/nginx/proxy-confs/*.subfolder.conf;

    # Reverse proxy for Open WebUI
    location / {
        proxy_pass http://open-webui:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
        ↪  $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        # Required for WebSockets
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 900s;
        proxy_send_timeout 900s;
    }

    # Reverse proxy for Ollama
    location /llm/ {
        auth_basic "Restricted Access";
        auth_basic_user_file /etc/nginx/.htpasswd;

        # If authentication is successful, forward the request
        ↪  to Ollama
        # The trailing slash is important, so that ollma does
        ↪  not see the /llm/ as part of the path!
        proxy_pass http://host.docker.internal:11434/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
        ↪  $proxy_add_x_forwarded_for;
```

System Hardening

It is recommended to put a proper firewall before the services we want to expose to the internet, but for our purposes we can just disable the port forwarding when we don't use the AI. It still makes sense to perform a minimal system hardening as shown in the following listing 18, before exposing the services to the internet.

```
# Install UFW
sudo pacman -Syu ufw
sudo ufw status

# Allow all outgoing connections
sudo ufw default allow outgoing

# Deny all incoming connections per default
sudo ufw default deny incoming

# Allow SSH for remote access
sudo ufw allow ssh

# Allow HTTP for certbot HTTP challange
sudo ufw allow http

# Allow HTTPS for Open WebUI
sudo ufw allow https

# Allow default Ollama port
sudo ufw allow 11434/tcp

# Make sure to run this command after adding the "allow" rules!
sudo ufw enable

sudo ufw status
```

Listing 18: UFW configuration

Expose Open WebUI and Ollama

For exposing the services from the home network via HTTPS to the internet, we need a domain name which points to the home router. To achieve this we can use DynDNS, which is provided for free by Infomaniak where the domain "hiube.ch"

was registered, and is supported by the FRITZ!Box used in the home network. The screenshots A.26 and A.25 are showing the respective configurations.

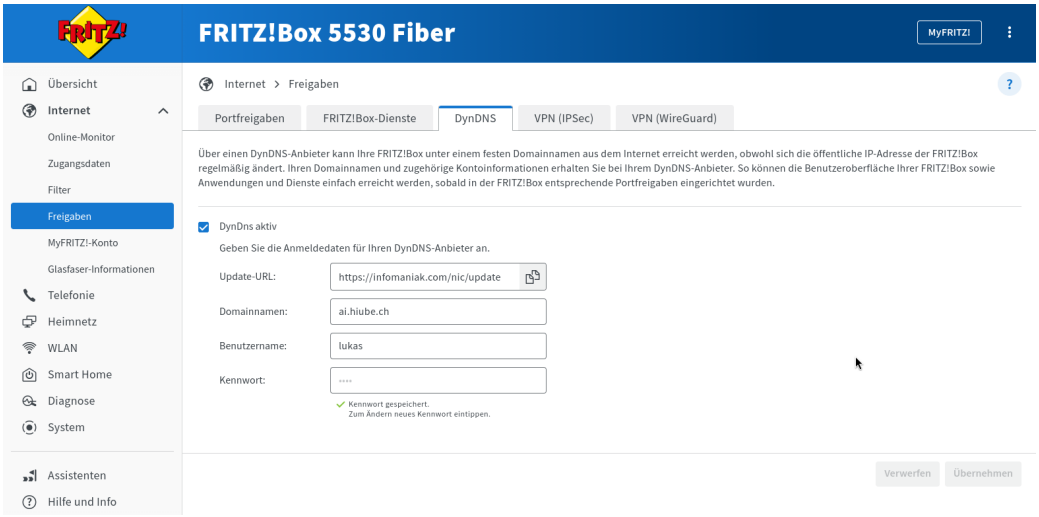

Figure A.25.: FRITZ!Box DynDNS Setup

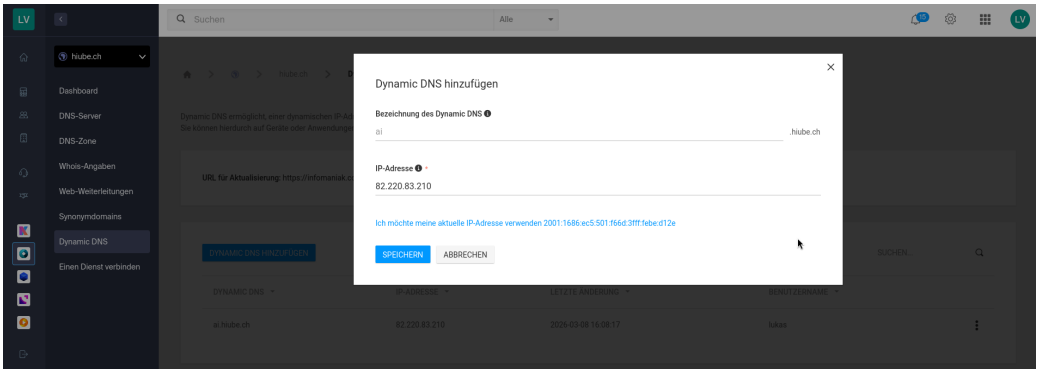

Figure A.26.: Infomaniak DynDNS Setup

Now that the domain name “ai.hiube.ch” points to the router in the home network, we need to redirect the HTTP(S) traffic to the LLM server. This can be achieved with port forwarding rules on the FRITZ!Box, as shown in the screenshot A.27.

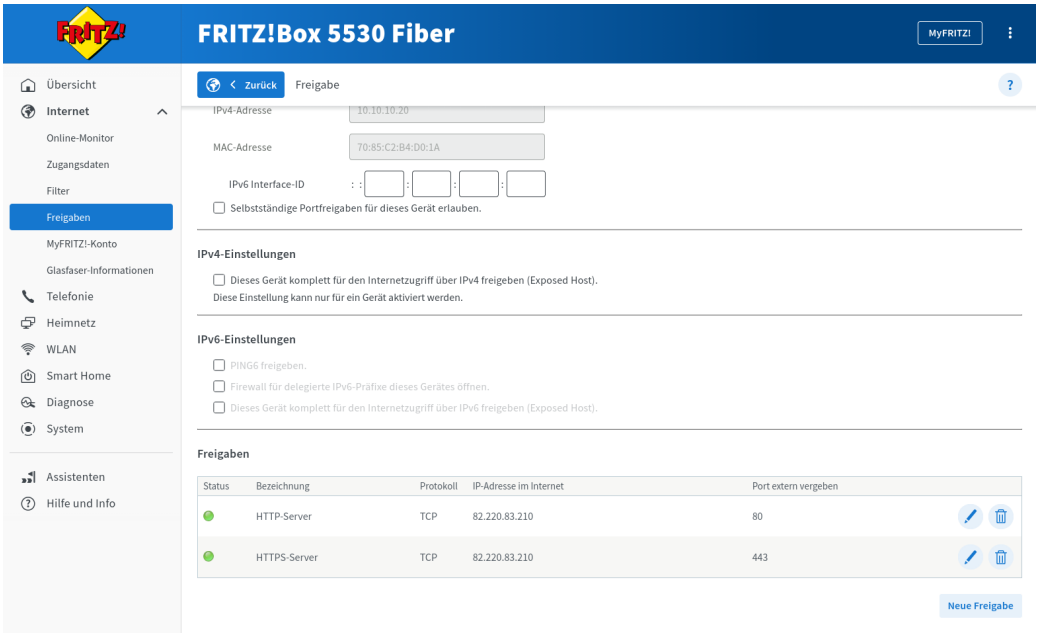

Figure A.27.: FRITZ!Box DynDNS Setup

The Open WebUI and Ollama API should already be reachable via “ai.hiube.ch”, but we will receive a security warning because there is no valid SSL certificate. The easiest way to generate the certificate, is to trigger certbot by restarting the “swag” service as shown in the listing below 19.

```
# Login as user ai and restart the swag service
podman-compose restart swag
```

Listing 19: Restart SWAG Container

A.12.3. Evaluation

For quickly evaluate models and assess their capabilities we can now use the Open WebUI served under “ai.hiube.ch”. The screenshot A.28 shows a quick test with “qwen2.5-coder:7B”.

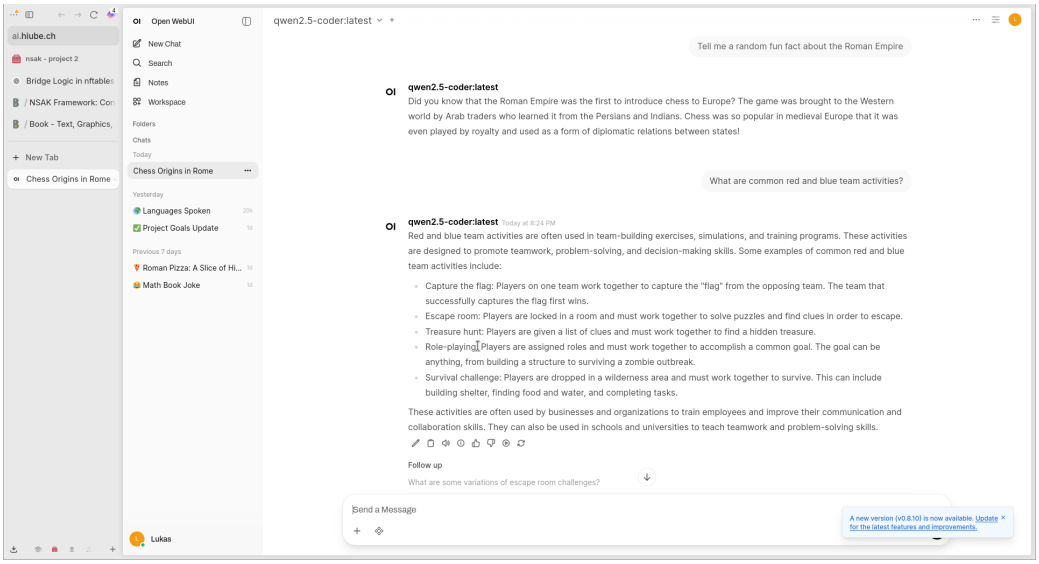

Figure A.28.: FRITZ!Box DynDNS Setup

Later on we can use the Ollama API served under `/llm/` as a backend for AI agents.

A.13. Writing Quality

Introduced from Professor David Stuckler the PEER-System: Simple and clear academic writing.

The structure is:

- P 1 POINT just one
- E Evidence and Example 1-n
- E Explanation
- R Repeat linking Links and leads to next topic